Surgical Inference: Decomposing Generative Models for Heterogeneous On-Device SoC Inference

Matt Mireles
Independent Researcher
mattmireles@gmail.com

October 21, 2026

Abstract

On-device generative inference usually means choosing one backend: CPU for compatibility, GPU for throughput, or an NPU when the compiler accepts the graph. Modern SoCs are heterogeneous, and single-backend deployment leaves most of the chip idle. We present *Surgical Inference*, a deployment methodology that changes the unit of compilation from "the model" to a set of fixed-shape neural submodels and native stages, each classified by computational motif and assigned to the SoC engine that measurement selects.

We evaluate it on two structurally opposite generative workloads. Kokoro-82M, a text-to-speech pipeline decomposed into four Core ML model families and three native Swift stages, is 1.8–2.4× faster than MLX, 2.7–6.5× faster than PyTorch MPS where MPS completes, and 3.2–6.8× faster than PyTorch CPU on three Macs. On an iPhone 15 Pro Max it synthesizes a 27 s clip in 1.7 s, 2.6–3.2× faster than the MLX Swift port. Magenta RealTime 2 (230M parameters), a live-music model with a 40 ms/frame streaming deadline, runs faster than real time on the same phone and yields three findings: the ANE admission cliff for streaming transformers is in-graph state mutation, not attention math; tensor layout decides FP16 numerical survival per compute unit; and per-call cost on weight-bound stages tracks weight bytes ÷ DRAM bandwidth.

Negative results are central. We could not produce a valid monolithic Core ML export of Kokoro: conversion fails at the data-dependent alignment, and the strongest partial monolith is numerically invalid. The ANE rejected Kokoro's largest dense stage on tensor geometry and every stateful variant of the music transformer, and the decomposed pipelines win anyway. The active ingredients are decomposition, static per-stage compilation, deliberate placement and validated host glue, not any single accelerator. Kokoro code, models and benchmarks are open source.

Keywords: on-device inference, heterogeneous computing, Apple Silicon, Neural Engine, Core ML, model decomposition, generative models, text-to-speech, music generation, autoregressive streaming, MLX

1 Introduction

The deployment of generative models on edge devices has emerged as a critical research direction, motivated by privacy, latency, network independence, energy cost, and user experience. The hardware is already heterogeneous. Apple Silicon integrates CPU cores, GPU cores, and a dedicated neural network accelerator (the Apple Neural Engine, or ANE) on a single die with unified memory. Similar CPU/GPU/NPU structures now exist across phones, laptops, tablets, and embedded systems.

Despite this hardware heterogeneity, the dominant approach to local AI inference on Apple Silicon uses Metal-based engines (Ollama/llama.cpp, MLX, PyTorch MPS) that target the GPU. That default is rational: the GPU is programmable, high-throughput, and forgiving. But it also creates a systems bottleneck. A GPU-only model competes with the user interface, rendering, media pipelines, and any concurrent local LLM. The neural accelerator remains mostly idle, CPU vector units are underused, and the application has fewer ways to trade latency, energy, and thermals.

The common alternative — compile the whole model to a neural accelerator — is too simple. Neural accelerators are fast and power-efficient for the graphs they admit, but their compilers impose static-shape, op-set, and tensor-geometry constraints. A modern generative model is rarely one homogeneous graph. It often contains dense feed-forward blocks, small recurrent or autoregressive components, data-dependent indexing, masking, DSP, sampling, and post-processing. Some of these parts are excellent neural-accelerator candidates. Some are better as native CPU code. Some should stay on GPU. Treating the model as a single scheduling unit hides this structure.

We argue that the deployment unit should be the *inference pipeline*, not the original model file. Surgical Inference decomposes a generative pipeline into smaller compiled submodels and native stages, then schedules them deliberately across the SoC. The procedure assumes only a heterogeneous SoC whose accelerators impose admission constraints, and applies when a model contains separable stages whose dataflow can be made explicit; the constraint catalog we document is Apple-specific, and all measurements in this paper are on Apple Silicon (§7.5). We measure it on two case studies chosen to stress different regimes. Text-to-speech (Kokoro-82M) is a clean batch example: recurrent prosody prediction, duration-dependent expansion, pitch/noise prediction, harmonic DSP, and dense convolutional waveform generation all coexist in one reference model. Live music generation (Magenta RealTime 2) adds the constraint the first lacks: a hard 40 ms autoregressive deadline that never pauses, on a phone, where a missed frame is an audible dropout rather than a slower benchmark row.

The systems objective is broader than raw latency. A decomposed pipeline can reduce wall-clock time, avoid GPU-only memory blowups, overlap CPU work with neural inference, and lower GPU residency so other workloads — including a local LLM — can run concurrently. This paper measures latency and completion behavior directly for both case studies, and — for the second — power and placement directly: paired Power Profiler captures and Instruments residency traces on physical iPhones (§6.7), including a thermal-soak result that currently fails and is reported as such. Shorter end-to-end execution, bounded memory use, and partial use of CPU/ANE instead of continuous GPU occupancy are the deployment properties that make lower-energy, cooler on-device inference plausible; the second case study turns part of that plausibility argument into measurement.

We introduce *Surgical Inference* and make five contributions:

1. **A decomposition methodology** for multi-stage generative pipelines on heterogeneous SoCs: a three-motif classification — Sequential-Dynamic, Data-Dependent Logic, Dense-Static — a decomposition procedure, and a per-stage placement ablation whose results overrode intuition in both case studies, in opposite directions (§3, §7.2). The methodology is a systematization of practice, not a new technique: per-model and per-stage Core ML packages already ship in Apple's Stable Diffusion conversion and in WhisperKit (§8). We make the classification, the placement ablation and the host-glue validation rules explicit, and test them on two opposite workloads.

2. **Two complete implementations with cross-device empirical validation.** A Kokoro-82M pipeline — four Core ML model families, three native Swift stages, zero Python at inference time — benchmarked on three Macs (M1 Mini, M2 Air, M2 Ultra) and two iPhones (A14, A17

Pro) against MLX, PyTorch MPS, PyTorch CPU, and two independent Core ML Kokoro ports (§4–5); and a Magenta RealTime 2 streaming pipeline holding a 25 Hz autoregressive deadline on a phone (§6).

3. **Negative results, reported as results.** We could not produce a valid monolithic Core ML export of Kokoro-82M with coremltools 8.3: conversion fails at the data-dependent alignment and the strongest partial monolith is numerically invalid (§5.9). The vocoder is rejected by the ANE on tensor geometry (`ANECCompile() FAILED` on every Mac tested). `.all` falls back silently on macOS; on iOS it hard-failed on iOS 26.5 and runs on iOS 27.0.1. The GPU frameworks fail on our workloads: PyTorch MPS hits a 65,536-output-channel limit on the 15 s and 30 s inputs on every Mac, and MLX is killed by the memory watchdog (jetsam) on the 4 GB phone. And naive native DSP ports regress until engineered (a 12× hn-nsf speedup in an early port from eliminating protocol-dispatched RNG, §3.3).

4. **Three ANE admission findings** from the streaming case study: in-graph *state mutation* — not attention math — is the compile-time cliff for streaming transformers, located by a falsification ladder and escaped by restructuring the transformer as a stateless step function (§6.3); tensor layout governs FP16 *numerical survival* per compute unit — after a channels-first rewrite the ANE was the only engine producing finite codec output (§6.4); and on weight-bandwidth-bound stages the ordinary roofline argument — per-call cost ≈ weight bytes ÷ DRAM bandwidth — holds for Core ML predictions on the phone, which reshaped the sampling loop from twelve predictions per frame into one (§6.5).

5. **Direct residency and energy instrumentation**, addressing what model-level scheduling claims usually leave unproven: Instruments Core ML traces showing per-model ANE hardware intervals and *zero* app-attributed Metal GPU intervals, turned into a machine-checkable placement-evidence artifact, plus paired Power Profiler captures showing the all-ANE policy removes process GPU impact and roughly halves CPU instructions versus a temporal-GPU control (§6.7). These traces cover the earlier all-ANE configuration; the corrected pipeline that ships today runs the temporal stage on the GPU and the depth stage on the CPU (§6.7).

All code, pre-converted Core ML models, and benchmark data for the first case study are available at:

- https://github.com/mattmireles/kokoro-coreml
- https://huggingface.co/mattmireles/kokoro-coreml

The second case study's conversion and validation layer is public at https://github.com/mattmireles/magenta-realtime-2-iphone — including the corrected exporters that reproduce the §6.3–§6.5 findings, the superseded exporters retained as negative-result artifacts, numerical-parity validators, reference fixtures, and validation receipts, with pre-converted packages mirrored on Hugging Face (generation status per artifact in Appendix A). The streaming runtime and instrumentation live in the Crossfade repository, a fork of magenta/magenta-realtime that is private at the time of writing; the full artifact and reproduction map is given in Appendix A.

2 Background

2.1 Heterogeneous SoCs as Inference Targets

Apple's M-series and A-series SoCs integrate three relevant compute units on a single die with unified memory architecture (UMA). All three compute units access the same physical memory

pool, which removes explicit device-to-device copy costs but does not make the compute units interchangeable. Each engine has a different instruction model, scheduling overhead, admission policy, and power profile.

CPU. Performance and efficiency cores optimized for sequential and branching workloads. Excellent at data-dependent control flow, dynamic shapes, and small-batch operations. The Accelerate framework provides vectorized DSP routines (vDSP) and BLAS operations.

GPU. Tile-based deferred renderer with compute shader support via Metal. Optimized for parallelizable workloads with flexible kernel programming through Metal Performance Shaders (MPS) and the MLX framework. Handles dynamic shapes and arbitrary computation graphs.

Neural Engine (ANE). Fixed-function matrix accelerator designed for high-throughput, low-power neural network inference. Accelerates a constrained subset of operations — convolutions, matrix multiplications, elementwise ops, pooling, normalization — under strict requirements:

- **Static shapes.** All tensor dimensions must be known at compile time.
- **Supported op set.** Operations outside the ANE-accepted set trigger fallback to CPU/GPU.
- **No data-dependent control flow.** Loops and conditionals depending on tensor values cannot execute on ANE.
- **Tensor geometry limits.** Per-axis dimensions are bounded (community documentation and our own compiler errors place the limit at 16,384 elements per axis); large intermediate activations that exceed ANE SRAM incur DRAM round-trips.

These constraints make the ANE ideal for dense, regular, feed-forward computation at moderate tensor sizes — but, as we show in §7.2, "dense and static" is necessary, not sufficient. Audio-rate tensors (hundreds of thousands of samples on the last axis) can categorically exceed ANE admission limits, in-graph state mutation can fail compilation outright (§6.3), and FP16 numerical health can depend on internal layout (§6.4). The practical inference target is therefore not "the ANE" or "the GPU"; it is the whole SoC, with stages mapped to engines according to what the compiler accepts and what measurement validates.

2.2 GPU-Centric Deployment and Its Blind Spot

The prevailing practice for local ML inference on Apple Silicon has converged on GPU-centric execution through Metal-based frameworks:

- **llama.cpp / GGML** [11]. The engine underlying Ollama and many local LLM tools. Uses Metal shaders directly. CPU and GPU only.
- **MLX** [13]. Apple's own NumPy-like framework for Apple Silicon. Purpose-built for the architecture but GPU-only; it cannot dispatch to the ANE.
- **PyTorch MPS** [5, 31]. Apple's Metal Performance Shaders backend for PyTorch. GPU only. Transparent fallback to CPU for unsupported operations.
- **Core ML** [3]. Apple's conversion and runtime framework. Capable of dispatching to CPU, GPU, or ANE. Used extensively for small on-device models but rarely for generative workloads. Its scheduler partitions graphs across compute units opaquely, and rejected ANE subgraphs fall back *silently* — a model "running on the ANE" may not be.

The conventional wisdom — reinforced by LLM-dominated discussion in the on-device ML community — holds that Core ML provides no meaningful benefit for generative inference. This is often true for large autoregressive language models: they are memory-bandwidth-bound, and the ANE does not create additional memory bandwidth beyond unified memory. But that conclusion does not transfer to all generative systems. Many models are not one uniform transformer loop. They are pipelines with separable dense, sequential, and data-dependent regions. GPU-only deployment collapses these regions back into a single queue on the GPU, monopolizing the most generally useful accelerator even when other SoC engines could do part of the work.

2.3 Kokoro-82M: The First Case Study

Kokoro-82M [14, 15] is a lightweight text-to-speech model released by hexgrad under Apache 2.0, based on the StyleTTS 2 architecture [22]. Despite only 82 million parameters, it achieves quality competitive with much larger models in community evaluations by leveraging efficient StyleTTS 2 components and an iSTFTNet-based decoder [18].

We use Kokoro-82M as a case study because it is small enough to ship on phones, high quality enough to matter, and structurally heterogeneous. The reference inference path contains seven conceptual stages:

1. **Duration prediction.** A BERT-based text encoder feeds a prosody predictor (LSTM + CNN stack) that produces per-phoneme durations, text encodings, and style features.
2. **Duration-dependent expansion.** Predicted durations expand token-level features into acoustic frames.
3. **Prosody (F0/N) prediction.** A small convolutional network predicts pitch (F0) and noise (N) contours from aligned features and style embedding.
4. **Decoder pre-processing.** The decoder's encode and decode blocks (convolutional residual stacks with style-conditioned AdaIN normalization) produce pre-vocoder features.
5. **Harmonic source generation (hn-nsf).** A SineGen-based module converts the F0 contour into a harmonic waveform, then applies STFT to produce spectral features used by the vocoder.
6. **Vocoder (GeneratorFromHar).** The final convolutional stack combines pre-vocoder features with harmonic features and produces the output waveform via iSTFT.
7. **Trim.** Output is trimmed to the target audio length.

The original PyTorch reference implementation executes all seven stages on CPU or GPU/MPS through one eager runtime, with significant Python overhead between stages. Surgical Inference asks a different question: which of these stages should remain a model, which should become native code, and which SoC engine should each stage use?

2.4 Magenta RealTime 2: The Second Case Study

Magenta RealTime 2 (MRT2) [23, 25] is Google's open-weights live-music model. Unlike a TTS model that renders a finished utterance, MRT2 *streams*: a decoder-only transformer autoregressively generates one frame of 12 residual-vector-quantization (RVQ) tokens every 40 ms, and a SpectroStream codec decoder — in the RVQ lineage of SoundStream [36] — converts each token frame into 1,920 samples of 48 kHz stereo PCM. Generation is steerable in real time by text, audio,

and MIDI conditioning. The upstream project ships real-time inference through MLX — GPU-only — and supports it exclusively on M-series Macs; there is no phone target.

We use the smallest configuration, `mrt2_small` (230M parameters), as the second case study because it is the structural complement of the first. Kokoro is feed-forward and batch: the deployment metric is wall time for a finished artifact, and every stage runs once per utterance. MRT2 is autoregressive and streaming: frame *N+1* cannot start before frame *N*'s tokens exist, the audio clock never pauses, and the deployment metric is sustained p99 frame time against a 40 ms deadline. It also exercises the one motif the first case study could route around: a stateful transformer with per-frame K/V-cache updates on the hot path (41-frame local-attention windows across 12 temporal layers, plus a 12-level serial depth sampler per frame). If the methodology only works when the Sequential-Dynamic motif can be left on the CPU, this workload exposes that.

3 Surgical Inference Methodology

Surgical Inference is a deployment-time graph partitioning method. It does not change model weights or retrain the network. It changes the compiled artifact boundary. A single reference model becomes a small number of fixed-shape neural packages plus native kernels for dynamic or irregular work. The method is intentionally simple:

- Cut only at real dataflow boundaries.
- Preserve numerical equivalence at each cut.
- Let measured stage placement override intuition.
- Optimize the host glue because it becomes part of the model.
- Use concurrency only where the data dependencies prove it is safe.

3.1 Computational Motifs

We classify stages into three motifs, each with a characteristic hardware affinity:

Motif 1: Sequential-Dynamic (SD). Characterized by recurrent layers (LSTMs, GRUs), autoregressive decoding with KV-cache, sampling loops, beam search, or computations where step t depends on step $t-1$. Tensor shapes may vary across inputs. Examples: LSTM-based duration prediction in Kokoro, autoregressive transformer decoding in LLMs, diffusion samplers with stepwise scheduler state.

Hardware affinity: **CPU or GPU** — as inherited. But the motif is a property of the boundary you draw, not the stage you inherit: when the sequential dependency is only cache state, the per-step math is Dense-Static and the recurrence is Data-Dependent Logic that can move to the host. Case study 2 shows this decomposition-within-a-stage unlocking ANE compilation for a streaming transformer (§6.3).

Motif 2: Data-Dependent Logic (DDL). Computations whose *structure* depends on tensor values, not just their shapes. Examples: duration-dependent expansion, top-k filtering, token suppression, dynamic masking, bucketing, ragged sequence packing, audio trimming, or branchy post-processing.

Hardware affinity: **CPU (native Swift/C).** Too small and too irregular for GPU or ANE dispatch to amortize. Best implemented directly, not through a tensor runtime.

Motif 3: Dense-Static (DS). Large blocks of convolutions, matrix multiplications, attention with fixed shapes, normalization, upsampling, or elementwise ops with shapes fully known at compile time and no data-dependent branching. Examples: convolutional vocoders, image decoders, denoisers, fixed-shape encoders, codec decoders, and BERT-style encoders with fixed input sizes.

Hardware affinity: **Core ML, with placement determined empirically.** Dense-Static work is the candidate set for ANE acceleration, but the ANE's admission constraints (op set, tensor geometry limits, SRAM budget) are checked at compile time per stage. A Dense-Static stage that the ANE rejects still benefits from Core ML compilation — fused fp16 kernels and ahead-of-time graph optimization on CPU+GPU — relative to eager-mode GPU frameworks. Placement is a measurement, not an assumption.

3.2 Decomposition Procedure

Given a generative pipeline, apply the following procedure:

1. Trace or instrument the reference forward pass and identify real dataflow boundaries:
   - values whose shape or semantics are stable enough to validate
   - boundaries where data-dependent logic enters or leaves the graph
   - independent branches that can run concurrently
2. For each stage, classify its computational motif (SD, DDL, DS).
3. For stages classified as Dense-Static:
   - Enforce static shapes via bucketing if necessary
   - Export as a Core ML `.mlpackage`
   - Validate numerical equivalence against a native-length fp32 PyTorch reference, across bucket fills (we score SNR; per-stage correlation on padded inputs proved too weak a gate, §4.2)
   - Ablate compute-unit placement per stage: try `.cpuAndNeuralEngine`, `.cpuAndGPU`, and `.all`; keep the fastest placement that passes the task validation gate. Do not trust `.all` to mean "ANE": rejected subgraphs fall back silently.
4. For stages classified as Data-Dependent Logic:
   - Implement natively in Swift (or C)
   - Use Accelerate/vDSP for vectorizable DSP
   - Profile the native implementation: a naive port is often slower than PyTorch until optimized
5. For stages classified as Sequential-Dynamic:
   - If the recurrence is only cache state, restructure as a stateless step function: caches as ordinary inputs, current-step updates as ordinary outputs, host owns mutation and continuity. The remaining graph is Dense-Static; classify it as such (§6.3)
   - If small, export as Core ML and measure CPU/GPU/ANE-adjacent policies anyway; the compiler may fuse useful subgraphs
   - If large and memory-bandwidth-bound, expect per-call cost $\approx$ weight bytes $\div$ DRAM bandwidth whichever compute unit runs it, since all of them share one DRAM; minimize weight re-streaming (fewer calls, lower precision) before chasing placement (§6.5)

- If latency is dominated by host dispatch, prefer fewer calls

6. Orchestrate the pipeline in Swift, with explicit model loading, tensor handoff via `MLMultiArray`, and inline timing of each stage. Profile the host-side glue: tensor materialization and padding are real costs. Where the dataflow allows, run CPU stages concurrently with Core ML predicts — heterogeneous units can genuinely overlap, and the decomposition makes the independence explicit.

7. Benchmark the assembled pipeline against:

   - The original PyTorch baseline (CPU and GPU/MPS)
   - GPU-native frameworks (MLX)
   - Any existing Core ML implementations of the same model
   - A monolithic conversion of the full model, if one can be built at all — the attempt itself is evidence (§5.9)
   - Across a range of input lengths and target hardware
   - With power/thermal instrumentation when energy claims are made

Step 3's ablation clause is load-bearing. In the first case study it changed the shipped configuration: only DecoderPre ships on the ANE, as a single policy for every machine (§7.2). Per-machine sweeps show the ANE also wins some smaller stages on some machines — on an M1, the Duration buckets from t128 up and the F0Ntrain buckets up to t600, by a few milliseconds each — but we ship one policy everywhere for simplicity. In the second case study the same clause ran the other way — measurement moved stages *onto* the ANE that the motif taxonomy had written off (§6.3) and revealed that for one stage the ANE was the only numerically correct placement (§6.4). A paper that merely says "we used the Neural Engine" would be wrong in both directions; the correct statement is that each pipeline uses the whole SoC deliberately.

3.3 CPU-Side DSP: A Non-Obvious Optimization Challenge

A subtle consequence of the decomposition procedure: when a stage is classified as DDL or must remain on CPU due to Core ML incompatibility, the naive Swift implementation is often *slower* than the PyTorch CPU reference. PyTorch's underlying numerical libraries are highly optimized; a direct Swift port without equivalent optimization will regress.

We observed this acutely with the hn-nsf harmonic source module. In an early port, the Swift implementation ran at 166 ms for 3 s of audio — nearly 4× slower than PyTorch. Profiling identified per-sample Gaussian RNG, dispatched through Swift's `RandomNumberGenerator` protocol, as the dominant cost (154 ms across 648k calls). Replacing this with a bulk pre-allocated noise buffer using direct xorshift64 reduced that early port's hn-nsf to 14 ms — a 12× speedup with identical numerical output. Subsequent vDSP vectorization of the noise/mask path and a frame-rate source-phase reformulation reduced it further; in the October 2026 pipeline hn-nsf costs 4.3 ms at the 3 s bucket and 40 ms at the 30 s bucket on M2 Ultra (§5.7).

This observation generalizes: **any Swift DSP port must be profiled and optimized with the same rigor as neural network inference.** Use Accelerate/vDSP for vectorizable operations. Use double precision for quantities that accumulate (phase, in cumulative sine generation). Pre-allocate buffers. Eliminate protocol dispatch in hot loops. The methodology explicitly incorporates this optimization pass; skipping it produces pipelines that are worse than the PyTorch baseline they are meant to replace.

3.4 Host-Side Materialization Cost

Decomposing a pipeline across multiple Core ML models and Swift stages introduces host-side overhead: model dispatch, tensor format conversion, padding to bucket geometry, and Swift-level control flow between stages. This cost is not negligible and it is easy to get wrong. Two host-side bugs in early versions of the Kokoro pipeline each cost more than any model stage: a one-hot alignment expansion implemented as a dense matrix multiply through zeros (later made sparse, and ultimately eliminated — the shipped code expands token vectors directly into frame buffers with no matrix at all), and boxed `MLMultiArray` element reads over a strided `Float16` waveform during trim. Both were invisible to model-level profiling and were found only by timing the full synthesize call. The methodology therefore times the *end-to-end* wall clock as the primary metric, with per-stage timings as diagnostics.

The second case study escalates this from a performance concern to a correctness concern: host glue that misreads strided Core ML outputs, or feeds a stateful graph inputs it silently ignores, can pass every model-level validation and still destroy output quality (§6.6). The procedure's validation step therefore extends past the graph boundary — every host-side consumer of a model output must be validated against the reference at least once, and stateful graphs need cross-prediction tests proving the state is actually read.

4 Case Study 1: Kokoro-82M Pipeline

4.1 Applying the Procedure

We apply the decomposition procedure from §3.2 to Kokoro-82M. The resulting pipeline is shown in Figure 1.

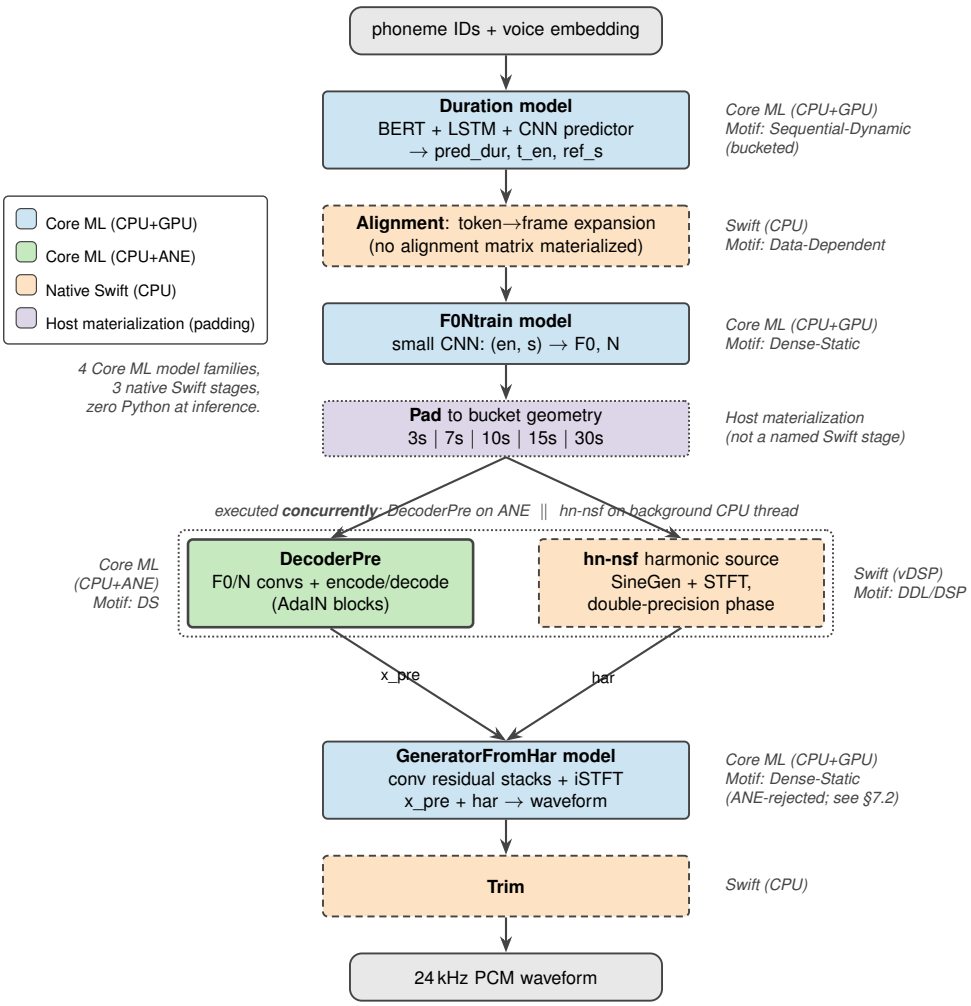


Figure 1: The decomposed Kokoro-82M pipeline: four Core ML model families and three native Swift stages, zero Python at inference time. Each stage is classified by computational motif and placed on the SoC engine that measurement selects (color = placement); only DecoderPre ships on the ANE, while GeneratorFromHar (geometry-rejected), F0Ntrain and the sequential Duration model run Core ML on CPU+GPU. The decomposition also exposes cross-unit parallelism a monolithic graph cannot express: DecoderPre (ANE) and the hn-nsf harmonic source (background CPU thread) execute concurrently.

Four Core ML model families, three Swift stages, zero Python at inference time. The per-stage compute-unit annotations above are the shipped *staged* policy, hard-coded in the pipeline's model loading: DecoderPre loads with `.cpuAndNeuralEngine`; Duration, F0Ntrain, and GeneratorFromHar load with `.cpuAndGPU`. The same policy applies on every machine. Section 7.2 explains why it is not an all-ANE plan.

Two structural points the diagram encodes. First, the alignment "matrix" no longer exists in the hot path: predicted durations drive a direct token-vector-to-frame expansion, so neither the one-hot matrix nor the matrix multiplies (`en = d @ align`, `asr = t_en @ align`) are ever materialized (the matrix construction survives only as a debug-dump path). Second, DecoderPre and hn-nsf execute *concurrently*: the harmonic source depends only on the padded F0 contour, so it runs on a background CPU thread while the DecoderPre Core ML predict is in flight, and

the pipeline accounts for the overlap explicitly in its stage timings. The decomposition does not just place stages on the right compute unit — it exposes cross-unit parallelism a monolithic graph cannot express.

For texts longer than one bucket, a caller-side chunking layer splits input at a fixed token cap and joins the per-chunk PCM; the benchmarks in §5 time single synthesize calls.

4.2 Core ML Export Details

The published Core ML models were exported with `coremltools` 8.3.0 [3, 4] and PyTorch 2.6.0, with `compute_precision=FLOAT16`. Key export decisions:

- **Duration model** is exported at enumerated token sizes (32, 64, 128, 256, 320, 384, 512) to accommodate inputs up to 30 seconds. Flexible shapes do not suit this stage or F0Ntrain: the ANE rejects flexible shapes, an `lstm` over a symbolic length runs about 5× slower on the CPU (51 vs 10.8 ms at 1,200 frames), and `EnumeratedShapes` costs about 140 ms on every call whose shape differs from the previous one. Both stages therefore keep per-size packages with masks. An environment-gated opt-in can load unpublished packages built for one exact token length; the library default, and every benchmark in §5, uses the padded mask-aware packages. Padded duration graphs were once expensive: in an early benchmark configuration the padded duration stage alone cost 82–733 ms per call on M2 Ultra, because its bidirectional LSTM was unrolled into up to 134,397 ops. Replacing the unrolled LSTM with two stock `lstm` ops (989 ops at t512) brought the padded stage to 11–45 ms on M2 Ultra (§5.7).

- **F0Ntrain, DecoderPre, and GeneratorFromHar** are exported at five bucket sizes targeting 3 s, 7 s, 10 s, 15 s, and 30 s of audio (F0Ntrain at frame counts t120/t280/t400/t600/t1200). The bucket is selected at runtime as the smallest that fits the predicted duration. GeneratorFromHar also ships as one flexible-length program: its time axis is a bounded `RangeDim` (40–2,400 `x_pre` frames), it runs on the GPU at the real length rounded up to a 0.5 s granule, and it keeps the mask. It is the runtime default on OS releases before 27; on OS 27 the runtime uses the fixed-bucket generators instead (§5.1).

- **AdaIN compatibility.** Early decoder-only exports failed due to MIL broadcast issues with AdaIN layers, forcing IdentityAdaIN workarounds that degraded quality. Re-export with the decomposed DecoderPre model succeeded with full AdaIN and correlation 1.000000 vs PyTorch — suggesting the earlier failure was graph-size dependent rather than an AdaIN-specific issue.

- **Numerical validation.** Earlier packages were validated by per-stage correlation (> 0.9999) on padded test inputs. That gate proved too weak: bucket padding contaminated normalization statistics while per-stage correlation stayed high, and a blind perceptual evaluation found further defects it could not see (§7.5). Current packages are scored by SNR over the valid region against fp32 PyTorch of the same masked export wrapper, on real tensors from the frozen inputs: F0Ntrain 66.6–71.0 dB, DecoderPre `x_pre` 54.5–58.7 dB, fixed-bucket generators 46.1–46.8 dB, flexible generator 44.4–45.1 dB. The masking itself was checked by forcing one utterance through every bucket and scoring each fill against a native-length fp32 PyTorch run. The benchmark harness additionally enforces a waveform health gate.

4.3 Swift Stages

- **Alignment** replaces PyTorch's `repeat_interleave` → one-hot → matmul chain with direct expansion: each token's hidden vector is copied into its `pred_dur[i]` frame slots, in both token-

major (`d` → `en`) and channel-major (`t_en` → `asr`) layouts, with raw-pointer fast paths for contiguous `float32` arrays. This evolved in two steps — a dense matmul (the §3.4 bug), then a sparse formulation, then no matrix at all.

- **hn-nsf** implements SineGen (harmonic sine wave generation from F0) and STFT natively using Accelerate/vDSP. Key implementation details, each visible in the shipped source:
  - Double precision (`Double`) for phase accumulation; single precision (`Float`) for output. Phase drift compounds over hundreds of thousands of samples; double precision is load-bearing.
  - A *frame-source phase* fast path: the original port upsampled F0 by 300× and immediately downsampled the phase increments back — for the model's exact nearest-neighbor geometry these cancel, so the current code computes phase increments directly per F0 frame (waveform-preserving, with the legacy RNG draw order kept for bit-compatible noise).
  - Bulk noise generation into a pre-allocated buffer using xorshift64 directly, rather than Swift's `RandomNumberGenerator` protocol; the Gaussian noise and voiced/unvoiced mask paths are vDSP-vectorized.
  - The 20-point Hann-window STFT uses a precomputed windowed DFT basis (built once, cached) and vectorized decimated dot products, matching PyTorch's `center=True` replicate padding.
  - Runs concurrently with DecoderPre (§4.1), so its remaining cost is partly hidden behind the Core ML predict.
  - Checked against PyTorch's STFT of the same source: after the DC/Nyquist sign fix (§7.5), no valid frame of the frozen 15 s input differs at the Nyquist bin, down from 10,658 of 66,720.
- **Trim** slices the output waveform to the natural utterance length, reading the strided `Float16` output buffer directly; no post-processing follows.

The pipeline also pools and reuses input `MLMultiArray` buffers across calls (zeroed, then filled via `memcpy` over contiguous strides) rather than allocating fresh feature arrays per synthesis — part of the host-materialization discipline from §3.4.

5 Case Study 1 Evaluation

5.1 Experimental Setup

Hardware. Three Apple Silicon Macs spanning the consumer-to-workstation range (Table 1), plus two iPhones (§5.6):

Table 1: Case Study 1 evaluation hardware: three Apple Silicon Macs spanning the consumer-to-workstation range.

Machine	CPU	GPU	ANE	Memory	Bandwidth
M1 Mac Mini	8-core (4P+4E)	8-core	16-core	16 GB	68 GB/s
M2 MacBook Air	8-core (4P+4E)	10-core	16-core	24 GB	100 GB/s
M2 Ultra Mac Studio	24-core (16P+8E)	76-core	32-core	64 GB	800 GB/s

Software. Swift 6.x; PyTorch 2.5.0 for the PyTorch arms; coremltools 8.3.0. Every Mac result in §5.2–§5.8 comes from October 2026 runs on repository commit `d32ef0d3` with the published

model set at Hugging Face revision `dadebca4`. The M2 Air and M1 Mini rows come from one run per machine. The M2 Ultra rows come from a second, quiet overnight pass, because the first pass failed the load gate below; the one exception is the M2 Ultra soniqo ratio, which comes from the loaded first pass (§5.5). The M2 Ultra ran macOS 26.7.1 (25G241); the M2 Air and M1 Mini ran macOS 27.0.1 (26A434). All benchmarks run with the machine plugged in and background services paused.

Inputs. Five frozen text inputs targeting the five runtime buckets, all using the same voice preset (`af_heart`), speed = 1.0:

Table 2: Frozen text inputs and their target runtime buckets for Case Study 1.

Key	Audio duration	Bucket
3s	2.80 s	3s
7s	6.75 s	7s
10s	9.60 s	10s
15s	13.90 s	15s
30s	27.38 s	30s

Timing boundary. From immediately before the synthesis call until full PCM audio is materialized in memory — token IDs in, 24 kHz waveform out (Table 2 gives the five frozen inputs targeting these buckets; Appendix B gives the exact boundary shared across all comparison implementations). Process startup, model download, and playback are excluded. All reported numbers are *warm* medians: models preloaded and primed, first-use Core ML compile/cache effects discarded.

Configurations.

- **F. Surgical (this work).** The full Swift + Core ML pipeline: staged compute policy and the padded, masked duration packages (§4.2). Every Mac row uses the five fixed-bucket generators, which the runtime uses by default on OS 27. The flexible-length generator can abort the process on macOS 27.0.1 inside Apple's MPSGraph runtime (intermittently), so the runtime disables it on OS 27. On macOS 26 the runtime default is the flexible generator, so on the M2 Ultra (macOS 26.7.1) the benchmark forced the fixed buckets through its models directory and reports the flexible generator as a separate column.
- **MLX.** Blaizzy/mlx-audio [9] 0.4.3 at commit `862dfbe`, running `mlx-community/Kokoro-82M-bf16`. The default MLX deployment path for Kokoro.
- **A. Python HAR-post hybrid.** The prior production pipeline: Python/PyTorch prefix + Core ML vocoder. State of the art for Kokoro-specific Core ML optimization prior to this work.
- **D. PyTorch MPS.** End-to-end PyTorch with `device=mps` and `PYTORCH_ENABLE_MPS_FALLBACK=1`. The default Apple Silicon GPU deployment path.
- **E. PyTorch CPU.** End-to-end PyTorch with `device=cpu`. Floor baseline.

Configuration letters are stable identifiers inherited from the repository's benchmark harness so that published tables remain traceable to archived result files. Configs B and C — a naive decoder-only Core ML artifact under `.all` and `.cpuAndGPU`, from an earlier April 2026 bakeoff generation with a different input set — are superseded by the decomposed pipeline and omitted here.

On each machine the arms ran in a fixed order (F, MLX, the external ports, then A/D/E); on the 16 GB M1 Mini, A and D/E ran as separate invocations. Configuration F discards three preflight calls, records one cold call, and reports the median of ten warm calls; MLX and the external ports report the median of ten warm calls after one cold call; the PyTorch arms run five counterbalanced iterations (config and input order shuffled per repetition, `torch.mps.synchronize()` before stopping the MPS timer). Tables report medians; the raw per-iteration timings for every cell are archived in the repository (Appendix A).

Before each arm the harness waited up to 20 minutes for the one-minute load average to fall to 1.5. The M2 Ultra was re-run overnight and cleared the gate for every arm. The M1 Mini cleared it for every arm except Configuration F (load 2.3). The M2 Air did not clear it (load 2.2–2.9), so its rows are mild upper bounds.

5.2 Headline Results: Current Pipeline

Warm median end-to-end wall time, Configuration F, October 2026 (Table 3):

Table 3: Warm median end-to-end wall time (IQR), Configuration F (Surgical), fixed-bucket generator, $n = 10$, October 2026. The last column is the M2 Ultra with the flexible-length generator.

Input	Audio	M2 Ultra Studio	M2 Air	M1 Mini	M2 Ultra, flexible
3s	2.80 s	**44.5** (0.6) ms	123.4 (0.8) ms	205.3 (1.9) ms	44.7 ms
7s	6.75 s	**80.0** (0.9) ms	273.6 (3.3) ms	410.3 (2.4) ms	79.6 ms
10s	9.60 s	**111.8** (1.2) ms	397.2 (0.7) ms	589.8 (2.4) ms	109.9 ms
15s	13.90 s	**147.6** (1.5) ms	589.1 (7.2) ms	834.9 (9.8) ms	141.1 ms
30s	27.38 s	**286.5** (3.3) ms	1175.5 (9.4) ms	1635.5 (10.2) ms	271.8 ms

Real-time factor (audio duration ÷ wall time) is given in Table 4:

Table 4: Real-time factor (audio duration ÷ wall time) for Configuration F.

Input	M2 Ultra	M2 Air	M1 Mini
3s	63× RT	23× RT	14× RT
7s	84× RT	25× RT	16× RT
10s	86× RT	24× RT	16× RT
15s	94× RT	24× RT	17× RT
30s	**96× RT**	23× RT	**17× RT**

The M2 Ultra synthesizes the 27 s clip in the 30 s bucket in 287 milliseconds, and in 272 ms with the flexible generator. The M1 Mini — the least expensive Mac in our set — completes the same workload in 1.64 seconds, 17× faster than real-time. Every machine at every duration operates far above real-time, sufficient for streaming TTS with headroom for concurrent workloads. These rows use the padded duration packages the library ships by default; the fixed-bucket generator is the runtime default on the macOS 27 machines, while the M2 Ultra's macOS 26 default is the flexible generator in the last column. Against our June 2026 run, which used per-length exact-duration packages, the three Macs are 1.13–1.32× faster at every bucket; the OS also changed between the two runs, so we do not attribute the whole gain to the pipeline.

5.3 vs MLX

Same machines, same frozen inputs, same voice, same timing boundary, median of warm calls (Table 5); Figure 2 plots the same data:

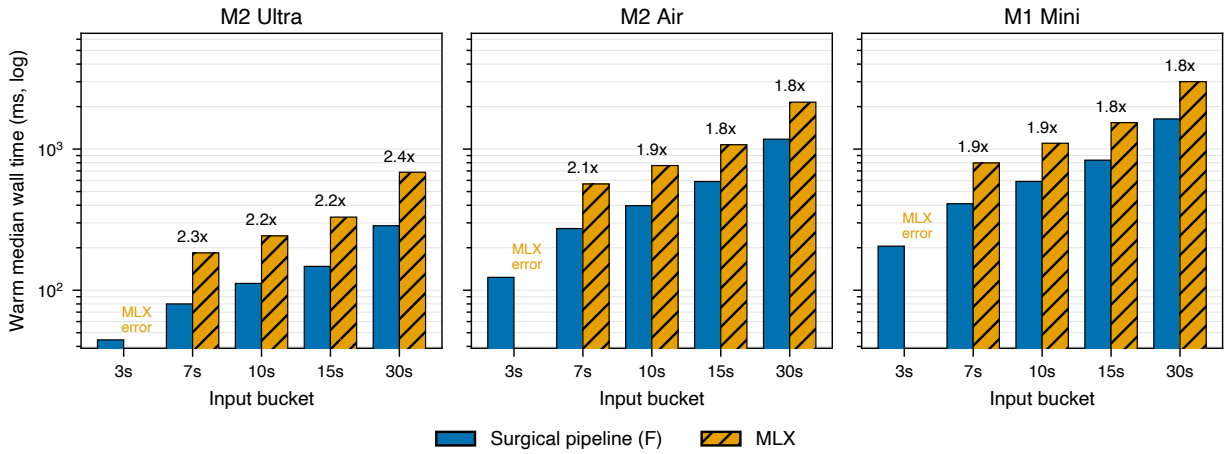


Figure 2: Kokoro-82M end-to-end latency, the Surgical pipeline (Config F) vs. MLX, warm medians across three Macs and five input buckets (log axis; §5.3). The Surgical pipeline is faster on every completed cell, by 1.8–2.4× (annotated). The pinned MLX build fails every 3 s input with a broadcast-shape error, so no 3 s comparison exists.

Table 5: Warm median wall time, Configuration F vs MLX, October 2026; MLX 3 s inputs fail with a broadcast-shape error.

Input	M2 Ultra (F vs MLX)	M2 Air	M1 Mini
3s	44.5 ms vs *error*	123.4 ms vs *error*	205.3 ms vs *error*
7s	80.0 vs 183.8 ms — **2.3**×	273.6 vs 566.0 ms — **2.1**×	410.3 vs 797.4 ms — **1.9**×
10s	111.8 vs 242.7 ms — **2.2**×	397.2 vs 764.5 ms — **1.9**×	589.8 vs 1100.3 ms — **1.9**×
15s	147.6 vs 329.7 ms — **2.2**×	589.1 vs 1073.8 ms — **1.8**×	834.9 vs 1540.0 ms — **1.8**×
30s	286.5 vs 685.3 ms — **2.4**×	1175.5 vs 2150.0 ms — **1.8**×	1635.5 vs 3003.7 ms — **1.8**×

The Surgical pipeline is faster on every bucket on every machine, by **1.8–2.4**×, with the gap widest on the M2 Ultra (2.2–2.4×) and nearly flat on the M2 Air (1.8–2.1×) and M1 Mini (1.8–1.9×). The pinned MLX version fails all 3-second inputs with a broadcast-shape error, so no 3 s comparison is reportable.

5.4 vs PyTorch

Warm median wall time, milliseconds, counterbalanced A/D/E run with Configuration F from the same session (Tables 6–8). The harness's PyTorch input set has four lengths, so these tables have no 10 s row. "err" marks cells where PyTorch MPS raised `Output channels > 65536 not supported at the MPS device` (torch 2.5.0) on every iteration.

Config F wins every cell where the competitor completes. Three observations:

1. **vs MPS: 2.7–6.5× where MPS finishes at all.** PyTorch MPS no longer completes the 15 s and 30 s inputs on any of the three machines: an op exceeds the MPS backend's 65,536

Table 6: PyTorch comparison, M2 Ultra (64 GB), October 2026.

Input	A (Python HAR)	D (MPS)	E (CPU)	F (Surgical)	F vs A	F vs D	F vs E
3s	124 ms	193 ms	259 ms	**45 ms**	2.8×	4.3×	5.8×
7s	222 ms	376 ms	530 ms	**80 ms**	2.8×	4.7×	6.6×
15s	387 ms	err	1010 ms	**148 ms**	2.6×	—	6.8×
30s	631 ms	err	1921 ms	**287 ms**	2.2×	—	6.7×

Table 7: PyTorch comparison, M2 MacBook Air (24 GB), October 2026. D completed 2 of 5 iterations at 3 s and 7 s.

Input	A (Python HAR)	D (MPS)	E (CPU)	F (Surgical)	F vs A	F vs D	F vs E
3s	236 ms	367 ms	615 ms	**123 ms**	1.9×	3.0×	5.0×
7s	616 ms	1765 ms	1460 ms	**274 ms**	2.3×	6.5×	5.3×
15s	945 ms	err	3023 ms	**589 ms**	1.6×	—	5.1×
30s	3381 ms	err	6114 ms	**1176 ms**	2.9×	—	5.2×

output-channel limit, and `PYTORCH_ENABLE_MPS_FALLBACK=1` (set for every run) does not route it to the CPU. (In our April 2026 run on older OS releases it ran out of memory on the 24 GB Air.) The "just use the GPU" default is not merely slower; it does not complete the longer workloads. The 6.5× Air cell rests on two completed iterations.

2. **vs CPU: 3.2–6.8×**, largest on the M2 Ultra and roughly flat across lengths on each machine.

3. **vs the Python+Core ML hybrid (A): 1.3–2.9×.** The narrowest margin is on the M1 Mini, where Config A's Core ML vocoder dominates both pipelines' wall time; Config F's advantage comes from replacing the Python prefix with Core ML + Swift and from eliminating host-side materialization costs.

5.5 vs Other Core ML Ports

The same run also benchmarked two independent Core ML Kokoro implementations on the same machines and inputs. Both comparisons require boundary caveats, which we state because they change the interpretation:

- **laishere/kokoro-coreml** (pinned `484907d`) runs a seven-package Core ML chain. Its public benchmark boundary *excludes* G2P and feed preparation, timing only the Core ML chain — narrower than our token-IDs-to-PCM boundary. On the M2 Ultra its chain-only times are 88 / 163 / 211 / 295 / 532 ms across the five buckets, and the Surgical pipeline is 1.9–2.0× faster on every bucket. On macOS 27.0.1 its converter aborts with the same MPSGraph assertion that disables our flexible generator (§5.1), so it has no M2 Air or M1 Mini rows. In our June 2026 run on macOS 15.7.7, its chain-only numbers were 1.1–1.3× faster than ours on the M1 Mini's short and medium buckets; that comparison cannot currently be repeated.

- **soniqo/speech-swift** (pinned `0d09a2e`) still emits a fixed ≈5.0 s audio artifact for every input of 7 s or longer, so its long-bucket timings are not full-duration synthesis and are not comparable. On the only comparable cell (3 s), the Surgical pipeline is 1.6× faster on the M2 Ultra (measured

Table 8: PyTorch comparison, M1 Mac Mini (16 GB), October 2026.

Input	A (Python HAR)	D (MPS)	E (CPU)	F (Surgical)	F vs A	F vs D	F vs E
3s	281 ms	553 ms	654 ms	**205 ms**	1.4×	2.7×	3.2×
7s	543 ms	1815 ms	1632 ms	**410 ms**	1.3×	4.4×	4.0×
15s	1058 ms	err	3235 ms	**835 ms**	1.3×	—	3.9×
30s	2066 ms	err	6487 ms	**1636 ms**	1.3×	—	4.0×

in a loaded session, against Configuration F from the same session; the overnight soniqo run failed), 2.0× on the M2 Air, and 1.8× on the M1 Mini.

5.6 On iPhone

The same model families deploy to iOS. The iPhone runs use the flexible-length generator, which ran correctly on iOS 27.0.1 on both phones, together with the padded duration packages and the staged compute policy. The shipped library currently disables the flexible generator on all OS 27 releases, iOS included, as a precaution against the macOS 27.0.1 abort (§5.1), and falls back to the fixed-bucket generators. Release-build results on iOS 27.0.1, October 2026, are given in Table 9. Each cell is one app launch running one arm on one bucket: 3 preflight calls discarded, 1 cold call, and the median of 10 warm calls. The phone cooled for at least 10 minutes before every launch, and arm order alternated by bucket.

Table 9: iPhone Release-build warm medians, iOS 27.0.1, October 2026: Configuration F (staged policy, flexible generator) vs the MLX Swift port (kokoro-ios 1.0.8). †MLX 15 s on the A17 Pro started at thermal state *fair* and ended *serious*, so its time is an upper bound. ‡Killed by the iOS memory watchdog (jetsam).

	iPhone 15 Pro Max (A17 Pro)			iPhone 12 Pro (A14)		
Input	F	MLX	F vs MLX	F	MLX	F vs MLX
3s	**195 ms**	543 ms	2.8×	**421 ms**	926 ms	2.2×
7s	**417 ms**	1198 ms	2.9×	**917 ms**	2050 ms	2.2×
10s	**609 ms**	1634 ms	2.7×	**1391 ms**	2809 ms	2.0×
15s	**881 ms**	2802 ms†	3.2×	**1984 ms**	4062 ms	2.0×
30s	**1721 ms**	4508 ms	2.6×	**4017 ms**	OOM‡	—

The Surgical pipeline beats the MLX Swift Kokoro port (mlalma/kokoro-ios [26] 1.0.8 — the Python mlx-audio above does not run on iOS) on every bucket on both phones: by **2.6–3.2**× on the A17 Pro and **2.0–2.2**× on the A14. The A17 Pro synthesizes the 27 s clip in 1.72 s (15.9× real time) and the A14 in 4.02 s (6.8×). Both phones run Configuration F about twice as fast as our July 2026 measurement on iOS 26.5 (A17 Pro 30 s: 3742 ms then, 1721 ms now), which used the fixed-bucket generator; the MLX arm moved far less, so the margin widened from 1.25–1.44× to 2.6–3.2×. OS and pipeline changed together, so we do not attribute the gain to either alone.

Two methodological findings from the July rerun still govern the protocol. First, Debug builds are not usable for cross-framework ratios: the MLX arm gained 1.4–1.6× from Release compilation while the Core ML arm barely moved. Second, on a fanless phone the run *order* is a thermal treatment, so each arm runs alone after a cooldown and the October run records thermal state on every call. The kokoro-ios API takes raw text, so its timings include a G2P pass ours excludes;

measured in isolation, that pass costs 0.17–1.05 ms per input — a disclosed asymmetry far below every observed margin.

On the 4 GB iPhone 12 Pro, MLX still cannot complete the 30 s input: the iOS memory watchdog kills it while loading weights, while the Surgical pipeline synthesizes the same input in 4.0 s. Bounded memory is a first-class constraint on phones — fixed-bucket and flexible Core ML synthesis both ran inside the 4 GB jetsam budget where the GPU framework did not.

The `.all` compute policy, which hard-failed on both phones on iOS 26.5 (Core ML error −9 at the duration stage), now runs the 3 s input on both phones on iOS 27.0.1 (200 ms on the A17 Pro, 412 ms on the A14). The library still ships the explicit staged policy, which the benchmarks above use.

5.7 Stage-Level Breakdown

Per-stage times for Configuration F, from the last warm call of each input in the October 2026 run, are given in Table 10. "Generator" is the Core ML total minus the three other model calls. hn-nsf runs on a CPU thread concurrently with DecoderPre, so its time overlaps DecoderPre's.

Table 10: Per-stage times, Configuration F with the fixed-bucket generator, last warm call per input, October 2026 (ms).

Machine	Input	Duration	F0Ntrain	DecoderPre	hn-nsf	Generator	Wall
M2 Ultra	3s	11.4	5.5	4.5	4.3	2.2	45.5
M2 Ultra	30s	44.9	19.4	51.9	40.0	52.1	288.7
M2 Air	3s	12.8	4.7	4.0	5.4	79.4	124.3
M2 Air	30s	70.3	27.3	32.7	48.7	888.5	1168.4
M1 Mini	3s	40.0	10.5	5.2	9.2	86.9	204.0
M1 Mini	30s	174.3	51.0	45.7	48.3	1083.0	1639.1

Three observations:

1. **On the M2 Ultra the pipeline is balanced; on base chips the generator dominates.** No stage on the M2 Ultra exceeds 53 ms at 30 s. On the M2 Air and M1 Mini the fixed-bucket GPU generator is 43–76% of wall time, consistent with their 10- and 8-core GPUs against the Ultra's 76. The next optimization target on consumer hardware is the generator, not the stages around it.

2. **The largest CPU stage is hidden behind the DecoderPre predict it overlaps.** hn-nsf cannot be effectively placed on Core ML (export attempts yielded correlation ≈ 0.00 due to SineGen's cumulative phase sensitivity), so the methodology routes it to optimized Swift DSP. After the §3.3 optimization pass it costs 40–49 ms at 30 s, and it runs on a CPU thread while DecoderPre (33–52 ms at 30 s) predicts.

3. **The gap between summed stages and end-to-end wall time is host-side work** — bucket padding, tensor materialization, dispatch — which is why the methodology treats full-wall-clock as the primary metric (§3.4).

5.8 Cross-Machine Scaling

Config F wall time relative to M2 Ultra is given in Table 11:

Table 11: Configuration F wall time relative to M2 Ultra, October 2026.

Input	M2 Ultra	M2 Air	M1 Mini
3s	1.0×	2.8×	4.6×
7s	1.0×	3.4×	5.1×
10s	1.0×	3.6×	5.3×
15s	1.0×	4.0×	5.7×
30s	1.0×	4.1×	5.7×

The gap widens with input length: at 3 s fixed per-call costs dominate on every machine, while at 30 s the generator's GPU work dominates on the base chips (§5.7). Per-bucket RTF is still nearly flat on each machine (§5.2). The load-bearing finding is that the Surgical pipeline remains comfortably real-time on the lowest-specification machine for inputs up to 30 seconds.

5.9 Control: We Could Not Produce a Valid Monolithic Core ML Export

The natural null hypothesis against decomposition is the monolith: convert the whole model to a single Core ML package, load it under one compute policy, and let the scheduler partition the graph. We tested it directly (coremltools 8.3.0, torch 2.6.0, M2 Ultra; full protocol and verbatim logs in the repository's experiment note, Appendix A). It fails at two independent seams — each one a boundary the decomposed pipeline already cuts:

- **The full token-IDs-to-waveform graph does not convert.** The duration-dependent expansion (`torch.repeat_interleave` driven by per-token *predicted* durations) has no static-shape lowering; `coremltools` aborts with `ValueError: Cannot add const [None]` while converting the op. `torch.jit.trace` appears to succeed only by baking one utterance's alignment into constants: the traced graph reproduces its own trace input at correlation 1.000 and hard-crashes on a same-shape input with different predicted durations (`The size of tensor a (65400) must match the size of tensor b (67200)`). It is a recording of one utterance's geometry, not a model.
- **The strongest partial monolith is numerically invalid.** Fusing every stage downstream of alignment into one package — aligned features to waveform, including the hn-nsf harmonic source and iSTFT — converts cleanly and produces wrong audio: correlation 0.27 (FP16) and 0.25 (FP32) against the PyTorch reference. FP32 being equally wrong rules out precision; eager-PyTorch cross-seed correlation of 0.996 rules out RNG mismatch. The divergence is structural, in Core ML's execution of the SineGen cumulative-phase path — the same failure that forced hn-nsf into native Swift (§4.3, §5.7).

There is therefore no single-package latency row to report. We could not produce a valid monolithic export with coremltools 8.3; with that toolchain, decomposition was not one deployment option among several but the move that gave us a correct, compilable on-device Core ML artifact at all. The comparisons in §5.3–§5.5 (GPU frameworks and other *decomposed* Core ML ports) are the strongest baselines we could build or find.

6 Case Study 2: Magenta RealTime 2 — Autoregressive Streaming Under a 40 ms Deadline

6.1 The Workload and the Real-Time Contract

MRT2's per-frame loop (§2.4) must complete in under 40 ms, every frame, indefinitely. The targets were fixed before any conversion work: p50 < 30 ms and p99 < 40 ms per frame, ≥ 10 minutes of sustained generation with zero audible dropouts, and sub-200 ms response to control changes. Note what changes relative to Case Study 1: there is no "bucket" to finish and no real-time *factor* to maximize. The success metric is a sustained zero-underrun stream, p99 against a wall-clock deadline, and thermal behavior over minutes — the regime where the ANE's power profile is supposed to beat the GPU.

Applying §3.2's classification to the per-frame graph (Table 12):

Table 12: Per-frame motif classification of the MRT2 graph.

Stage	Structure	Motif
Temporal transformer	12 layers, d=1024, 8 heads; 41-frame local-attention K/V caches (self + cross) per layer	Sequential-Dynamic
Depth transformer	2 layers, d=768; 12 *serial* RVQ levels per frame, each conditioned on the previous level's sampled token	Sequential-Dynamic
Sampling	temperature, top-k, Gumbel noise, per-level valid-range masks, RNG state	Data-Dependent Logic
RVQ detokenization	12 codebook gathers + summation	Data-Dependent Logic
SpectroStream decoder	17 conv2d + 7 conv2d_transpose, fixed shapes, tokens → STFT-rate features	Dense-Static
iSTFT / overlap-add	windowed synthesis to PCM, streaming carry	DDL / DSP
MusicCoCa prompt encoding	text → embedding → 12 RVQ style tokens → conditioning	off the hot path (control-rate)

One structural decision was made before any export and pays off throughout: **audio-rate tensors never enter a Core ML graph.** The codec decoder stops at STFT-rate features and host code (Accelerate) performs iSTFT and overlap-add. The Kokoro vocoder's fatal geometry problem (§7.2) — waveform-length axes that exceed ANE admission limits — is designed out rather than fought.

Setup. The same two iPhones as §5.6: iPhone 12 Pro (A14) and iPhone 15 Pro Max (A17 Pro). Every Case Study 2 measurement ran on iOS 26.5; the Case Study 1 iPhone runs above used iOS 27.0.1. Conversion went through minimal PyTorch wrappers reconstructed from the released safetensors and validated against the upstream MLX reference, then `torch.jit.trace` → `mlprogram`, FP16 except where noted, minimum deployment target iOS 18. Every export was gated

on numerical parity against MLX before any timing was trusted (transformer output correlations $>$ 0.9999; codec decoder SNR $\geq$ 115 dB; token-level sampling parity checked separately). Placement claims below come from `MLComputePlan` per-op reports and, for the composed runtime, Instruments traces on the physical phones — never from the requested compute-unit policy alone.

6.2 The Pipeline

Per frame, the shipped pipeline runs: host conditioning assembly → Core ML temporal transformer step → Core ML depth rollout (samples all 12 RVQ levels in one prediction from host-supplied noise; §6.5) → host RVQ codebook summation → Core ML codec decoder (overlapping 5- or 25-frame buckets) → host iSTFT/overlap-add → lock-free SPSC ring → `AVAudioSourceNode`. The pipeline is shown in Figure 3. The render callback makes one C call into the ring and never blocks on the model. Generation runs ahead of playback behind a PCM reservoir with high-watermark backpressure: the producer sleeps when the ring is full instead of generating unboundedly.

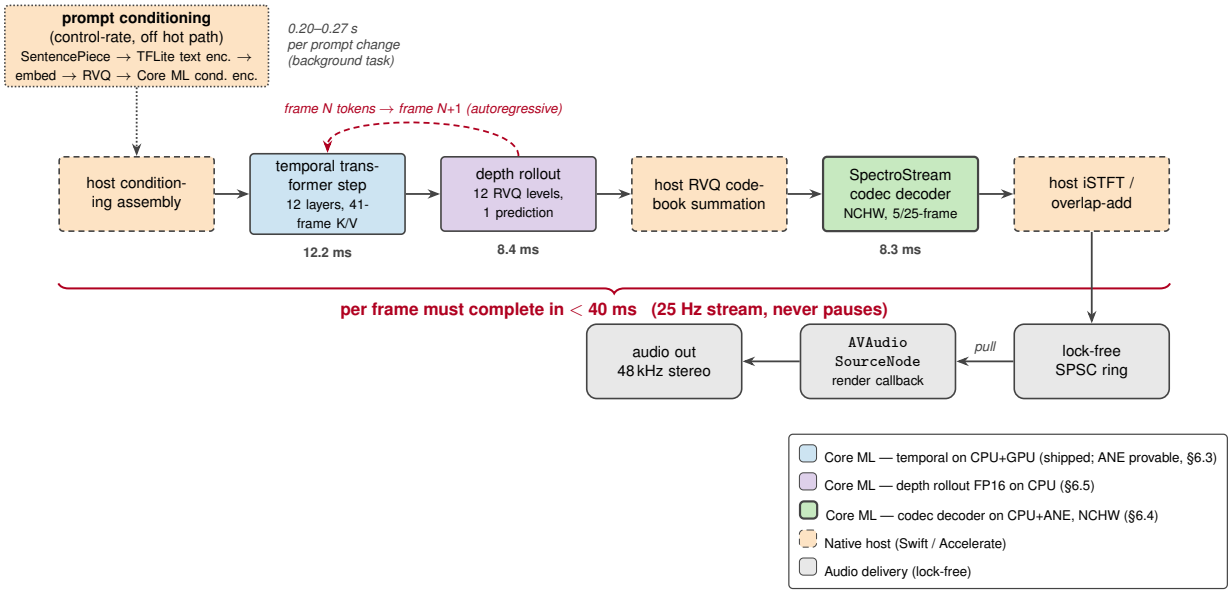


Figure 3: The Magenta RealTime 2 per-frame streaming loop. Every frame must complete within the 40 ms (25 Hz) audio deadline, which never pauses. Audio-rate tensors never enter a Core ML graph — the codec decoder stops at STFT-rate features and the host synthesizes PCM — and the three Core ML stages ship on three different engines (temporal on GPU, depth rollout FP16 on CPU, NCHW codec decoder on ANE; per-stage p50 on A17 Pro shown). Generation runs ahead of playback through a lock-free SPSC ring feeding an `AVAudioSourceNode`; the render callback never blocks on the model.

Prompt conditioning is a separate control-rate chain compiled entirely on device — SentencePiece tokenizer → TFLite text encoder → embedding mapper → RVQ quantizer → Core ML conditioning encoder — running in 0.20–0.27 s steady state per prompt change on both phones, on a background task off the render path. The chain is byte-identical across A14 and A17 Pro at every integer token stage, an unusually strong cross-device determinism property that made it possible to certify with fixture parity rather than statistical testing. A quantization footnote with a generalizable lesson: FP16 on the text encoder preserved all 28/28 test prompts' RVQ tokens byte-exactly (mapped-space drift 7×10^{-5}), while int8 dynamic quantization shifted tokens on 8/28

prompts (drift 2×10^{-3}) — **integer token boundaries downstream of an encoder are the real quantization gate**, not embedding-space error norms.

The per-stage placement story, unlike Case Study 1's, cannot be stated as one clean table — it is the subject of the next three subsections, and the honest answer differs between "what was proven possible" and "what ships today."

6.3 Finding: State Mutation, Not Attention, Is the ANE Cliff

The §3.1 taxonomy assigns the temporal transformer to CPU/GPU. The 25 Hz deadline made that assignment worth attacking, and the attack produced what we consider the paper's sharpest finding — with the caveat that the ladder below was measured on one phone, the A14.

The obvious route is Core ML's mutable state (`MLState`) holding the K/V caches. It fails in two ways. A one-frame stateful step *does* compile, with a real ANE island (`MLComputePlan`: ≈70% of estimated cost on ANE) — but every multi-frame unroll fails ANE compilation (`MILCompilerForANE ... ANECCompile() FAILED`, error −14): the full 2/4/8/16/25-frame matrix on the A17 Pro, and the 25-frame artifact on both phones. And the one-frame graph that compiles is not even a latency win under streaming pressure: 25 sequential `MLState` predictions on the A14 measure p99 29.7 ms per call, against 14.7 ms for the same graph CPU-only. A first host-owned-cache export — which still assembled cache updates in-graph — also failed ANE compilation. At this point the folklore conclusion ("streaming transformers don't fit the ANE") was fully available, and wrong.

Instead of concluding, we ran a falsification ladder: strip the graph to one suspect mechanism, prove it on device, add the next suspect back. Each rung is a separate `.mlmodelc`, measured on the iPhone 12 Pro with `MLComputePlan` placement evidence (FP16, p99 over warmed runs), summarized in Table 13 and Figure 4:

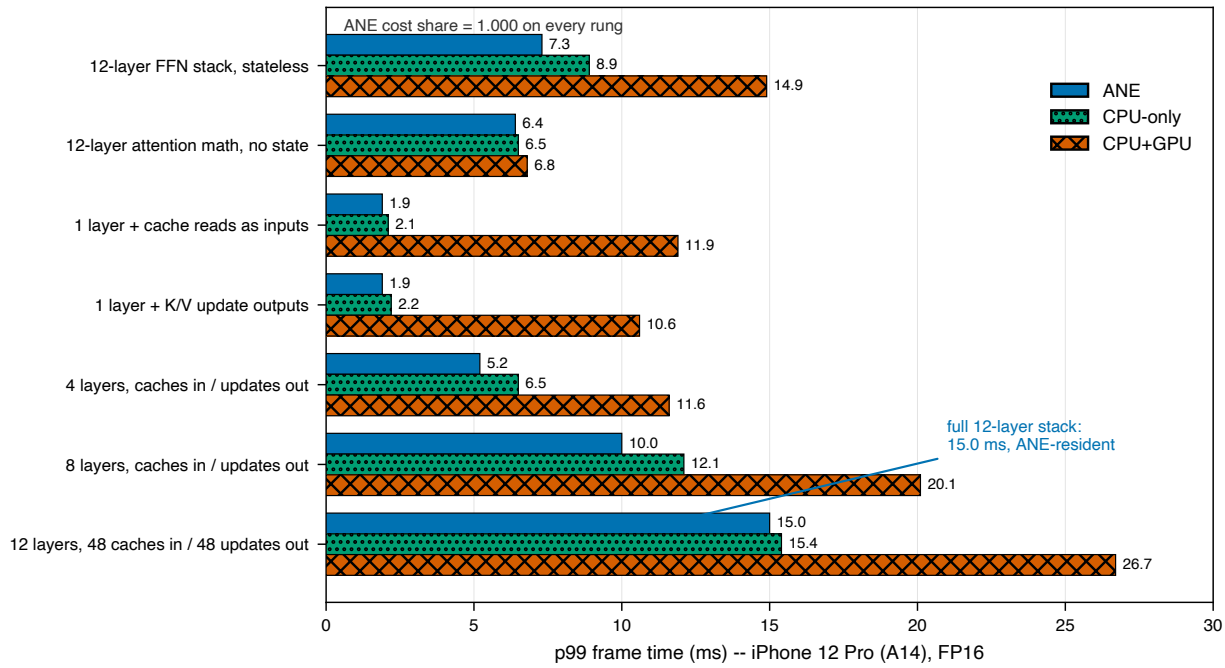


Figure 4: The falsification ladder that located the ANE admission cliff for streaming transformers (p99 per frame, iPhone 12 Pro / A14, FP16; §6.3). Every rung is ANE-clean (ANE cost share 1.000) and the ANE placement beats both CPU-only and CPU+GPU — including the complete 12-layer stack with all 48 K/V caches read in and 48 one-token updates written out, at 15.0 ms. The one thing that must be absent is in-graph cache mutation: attention, softmax, concatenation, and update outputs are all admissible; the state write is the cliff.

Table 13: Falsification ladder: p99 per rung across compute-unit policies, iPhone 12 Pro (A14), FP16; every rung ANE cost share 1.000.

Graph	ANE cost share	p99 ANE	p99 CPU-only	p99 CPU+GPU
12-layer FFN stack, stateless	1.000	7.3 ms	8.9 ms	14.9 ms
12-layer attention math, no state	1.000	6.4 ms	6.5 ms	6.8 ms
1 layer + cache reads as inputs	1.000	1.9 ms	2.1 ms	11.9 ms
1 layer + current-token K/V update outputs	1.000	1.9 ms	2.2 ms	10.6 ms
4 layers, caches in / updates out	1.000	5.2 ms	6.5 ms	11.6 ms
8 layers, caches in / updates out	1.000	10.0 ms	12.1 ms	20.1 ms
12 layers, 48 caches in / 48 one-token updates out	**1.000**	**15.0 ms**	15.4 ms	26.7 ms

Every rung is ANE-clean. The complete temporal stack — all attention, all FFN, all cache reads, all per-token cache-update outputs — compiles to a single ANE-resident graph (the only CPU ops are zero-cost casts) and beats both CPU-only and GPU placement, provided one thing is absent: *in-graph cache mutation*. Attention matmuls, softmax, concatenation against cache tensors, even forty-eight update outputs are all admissible; the cliff is the state write. Restated for the taxonomy: the recurrence in a streaming transformer is cache *bookkeeping* — Data-Dependent Logic that belongs to the host — wrapped around per-step math that is Dense-Static. Cut at

that boundary and the motif assignment inverts. Core ML becomes a stateless temporal-math accelerator; Swift/C++ owns cache storage, the shift-and-append, and continuity.

Honesty about where this stands in the shipped app: the cliff has teeth beyond compile time. The stateless-boundary artifact that ran cleanly in the headless runtime host (including a 10-minute zero-underrun streaming proof, §6.7) later hit a compiler failure with silent CPU fallback (≈640 ms/frame) inside the full application context, and the numerically *correct* rolling-cache stateful graph that replaced it during a correctness overhaul (§6.6) reproduces ANECCompile −14 on the A14 in both of its variants — both of which mutate caches in-graph, consistent with the finding. The shipped temporal placement is therefore `.cpuAndGPU` today, with the proven stateless boundary as the documented escape to re-land. Every graph that mutated cache state inside Core ML failed admission; every graph that did not, passed — but admission is also *instance-fragile*, and a placement that passed in a test harness must be re-proven in the shipping process.

6.4 Finding: Layout Determines Numerical Survival, Not Just Placement

The SpectroStream decoder is textbook Dense-Static: fixed-shape 2D convolutions and transposed convolutions. The naive FP16 export compiled — and produced non-finite output (finite ratio 0.71). The failure localized to the large parallel transposed-convolution upsampling tail; an FP32 variant was numerically clean but scheduled entirely onto the CPU at roughly 8× the latency the ANE would eventually deliver. The fix was neither precision nor op rewriting but *internal memory layout*: converting the upsampling block to channels-first (NCHW) internally while preserving the public channel-last I/O contract. The result, on the iPhone 12 Pro (FP16, `MLComputePlan` + on-device output-health checks), is given in Table 14:

Table 14: SpectroStream decoder placement and FP16 output health, iPhone 12 Pro.

Decoder artifact	Policy	p99	Output finite?
5-frame, channels-last FP16	any	—	**no** (0.71 finite ratio)
5-frame, NCHW FP16	`.cpuAndNeuralEngine` (ANE 1.000)	**6.7 ms**	**yes** (30,720/30,720)
5-frame, NCHW FP16	CPU-only	27.8 ms	no (29,056/30,720)
5-frame, NCHW FP16	CPU+GPU	32.2 ms	no (29,056/30,720)
25-frame, NCHW FP16	`.cpuAndNeuralEngine` (ANE 1.000)	**24.8 ms**	**yes** (184,320/184,320)
25-frame, NCHW FP16	CPU-only	152.7 ms	no

Two lessons. First, the placement win is large (≈4–6× over CPU) and total: the NCHW artifact schedules every executable op onto the ANE, including the `slice_by_index`/`conv_transpose` pattern that the channels-last version pushed to CPU. Second, channels-first internal layout is documented vendor guidance for ANE *throughput* [6]; what we have not seen documented is the numerical coupling: **the ANE was the only compute unit that produced finite output from the same FP16 artifact.** Placement and numerics are not independent axes. Our hypothesis for the mechanism is that each engine runs a different lowering of the same MIL program, with different accumulation and layout paths; we did not inspect those lowerings. What we measured is that FP16 survival differed across engines. "Validate numerically, per placement, on device" is not paranoia; for this stage it was the difference between music and NaNs.

This finding complements Case Study 1's op-rewrite null result (§7.2): there, source-level rewrites suggested by ANE folklore changed nothing because the binding constraint was tensor geometry; here, a layout rewrite changed everything because the binding constraint was the FP16 numerical path. The constant is that only a per-stage, on-device ablation tells you which regime you are in.

6.5 Finding: Weight Bandwidth Is the Invariant That Shapes the Graph

The depth transformer samples 12 RVQ levels per frame, serially — level k conditions on the token sampled at level $k-1$. The natural deployments are 12 small predictions per frame (full pass or stateful step). Both were built, both validated token-for-token, and both cost the same on device: ≈40–45 ms/frame on the A14 — over the entire frame budget — whether the call computes 12 positions or one.

The explanation is a single number. Every Core ML prediction streams the stage's full weights from DRAM: 97 MB (FP32 depth weights) ÷ 3.4–3.8 ms per call ≈ 26–29 GB/s — the A14's LPDDR4X bandwidth. Cross-check: the 364 MB FP16 temporal graph measures 13.9 ms isolated, the same ≈26 GB/s. **Per-call cost ≈ weight bytes ÷ DRAM bandwidth.** This is the ordinary roofline bound for a memory-bound stage. We did not sweep it per compute unit, but CPU, GPU and ANE share one DRAM, so no placement escapes it. Twelve calls per frame means twelve weight streams (≈1.2 GB/frame); no scheduler, compute unit, or state representation can fix that. It also reframes the ANE-vs-GPU question for bandwidth-bound stages: neither engine manufactures bandwidth, so latency parity is expected and the ANE's case rests on power (§6.7).

The fix that respects the invariant moves the 12-level autoregressive rollout *inside* one prediction, so weights stream once per frame — while keeping determinism host-owned, the methodology's standing rule for sampling. The host supplies per-level Gumbel noise `[12, 1024]` and inverse temperature as inputs (host RNG and seeds preserved; Gumbel-max [24] over a static top-k set is distributionally identical to top-k softmax sampling); top-k and valid-range masks are constants; the embedder feedback between levels is an in-graph gather reading 12 rows. Measured depth cost per frame: 37 ms FP32 → **12.7 ms FP16 on A14, 8.4 ms on A17 Pro**. Streaming the weights once per frame removed most of the cost, but the rollout does not reach the one-stream floor: 48.5 MB of FP16 weights at ≈26 GB/s implies ≈1.9 ms, well below the measured 12.7 ms. We have not decomposed the remainder. The FLOAT32 export is token-exact against the reference (0/900 mismatches across autoregressive chains); FP16 flips near-tie tokens without changing the distribution, and was shipped after instrumented and paired-listening gates.

This is the methodology absorbing a new rule rather than breaking: §3.2's "sampling stays on CPU" was always about *determinism and dynamism*, not geography. When bandwidth makes call count the binding constraint, sampling logic can move into the graph as long as the entropy source and the validation gates stay host-owned.

6.6 Finding: Host Glue Can Silently Destroy Correctness

Case Study 1's host-side bugs cost milliseconds (§3.4). This case study's cost music, and they motivate the validation rules now folded into the procedure. Three bugs shipped in early device builds while every model-level validation passed:

- **Strided output misread.** Core ML pads output rows for 64-byte alignment: the depth logits tensor `[1, 12, 12294]` carries strides `[147648, 12304, 1]` — 10 padding elements per row. The Swift sampler indexed `level × (count / 12)`, so level k read logits shifted by $10 \cdot k$ positions. Level 0 was correct (coarse structure always sounded plausible); levels 1–11 sampled from misaligned logit slots — scrambled fine detail, collapsed stereo. Every Python-side validation passed because `coremltools` honors strides; the Swift read path was never validated. Found by an RNG-free argmax A/B that diverged at frame 0, level 1, with matching logit *values* at offset indices.

- **Feedback that never existed.** The runtime allocated the temporal feedback input once,

zeroed, and never wrote it; and it made one depth prediction per frame where the contract requires 12 autoregressive levels. The transformer graphs were faithful; the loop around them did not implement the model.

- **Write-only state.** The one-frame stateful temporal graph baked an attention bias that masked *all 41 history slots*: it wrote K/V state every frame and never read it. Teacher-forced parity and within-prediction unroll parity both pass on a write-only state — the graph ran for weeks with no self-attention history before a fresh-vs-warmed-state bit-identity test exposed it.

The resulting process rules are now part of step 6 of the procedure: every host-side consumer of an `MLMultiArray` must be validated against a Python reference at least once, reads must go through `strides` rather than `count ÷ dims`, and a stateful graph is not validated until a cross-prediction test proves the state is *read* (same input on fresh vs. warmed state must differ, and an N-prediction stateful drive must match a streaming reference past the window size). After the fixes, the device output joins the reference cluster: left-right correlation 0.99 (matching the MLX reference), prompt-adherence embedding scores within the clean reference band, and blind automated listening gates passing at the shipped temperature with known-bad controls correctly rejected.

6.7 Placement, Power, and Residency Evidence

Case Study 1 stated ANE residency as policy intent (a limitation, §7.5). This case study instruments it directly, and the instrumentation is itself a contribution: requested compute units are a *request* (§2.2), and this workload produced multiple concrete demonstrations — `.cpuAndNeuralEngine` artifacts whose compute plan was 100% CPU, and an `.all` policy that mapped a stage entirely to the slower GPU.

Residency. Instruments Core ML traces on both phones, with the earlier all-ANE policy (before the §6.6 correctness fixes; the shipped placement now differs, see the end of this section), show per-model ANE hardware intervals for all three model families (on the A17 Pro: temporal 1,729 predictions averaging 7.8 ms of ANE time; depth 0.55 ms; decoder 18.9 ms) and a Metal GPU interval table containing *only* the system compositor — zero rows attributed to the app process. The check is encoded as a machine-readable gate: a placement-evidence artifact that requires ANE prediction intervals for every model family and rejects any app-attributed GPU interval, so future regressions fail a verifier rather than a vibe.

Power. Paired Power Profiler captures on the A14 — identical graphs and workload, only the temporal stage's compute policy varied — show the all-ANE policy at process `gpuImpact` 0.000 / `cpuImpact` 1.38 / 48.1 B CPU instructions over 60 s, against the temporal-GPU control at 2.23 / 2.63 / 110.3 B. The ANE policy removes process-attributed GPU impact entirely and cuts CPU instructions by 56%. In the same comparison the producer's *duty cycle* tells the thermal story: the ANE arm finished each second of audio fast enough to sleep 43% of the run under backpressure; the GPU arm worked 93% of wall-clock to deliver the same stream. (These captures predate the §6.6 correctness fixes; both arms ran the identical semantics, so the policy-controlled comparison stands, but the absolute throughput numbers do not transfer to the corrected pipeline.) On the A17 Pro, process-level power impact did *not* separate the policies — the Instruments residency trace was required to prove placement there, which is why the methodology treats power tables and residency traces as complementary rather than alternative evidence. Figure 5 plots the paired power metrics.

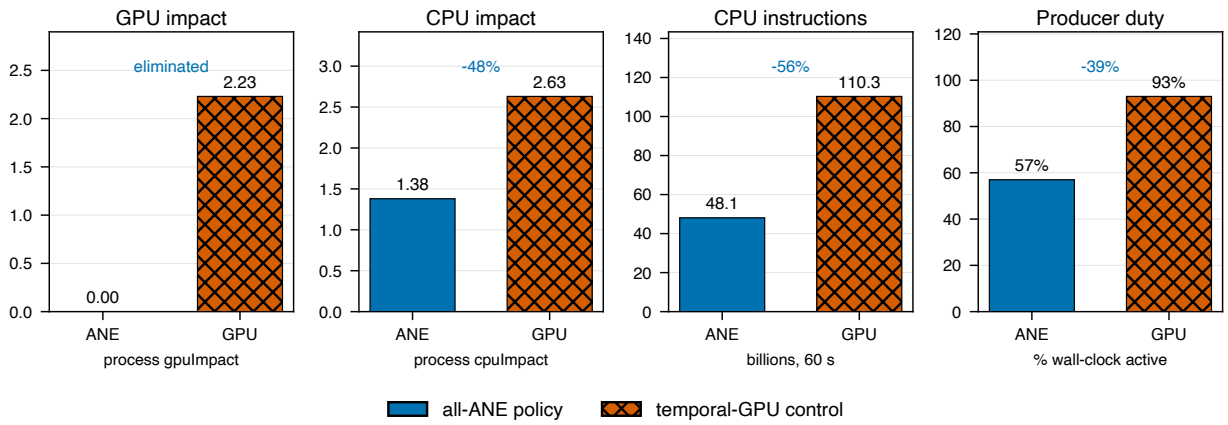


Figure 5: Paired power comparison on the A14 (iPhone 12 Pro), all-ANE temporal policy vs. a temporal-GPU control running identical graphs over 60 s (§6.7). The ANE policy eliminates process-attributed GPU impact entirely (2.23 → 0.000) and cuts CPU instructions by 56% (110.3 B → 48.1 B), while the producer's duty cycle drops from 93% to 57% — it finishes each second of audio fast enough to sleep 43% of the run under backpressure.

End-to-end status, stated plainly. With the corrected, numerically verified pipeline (rolling temporal on GPU, in-graph depth rollout FP16 on CPU, NCHW decoder on ANE): the iPhone 15 Pro Max runs faster than real time — composed ≈29 ms/frame p50 (temporal 12.2 + depth 8.4 + decoder 8.3), zero underruns over bounded 45 s runs with backpressure engaged, ≈1.37× real-time headroom. The iPhone 12 Pro does not currently hold the budget (≈50 ms/frame; combined temporal+depth DRAM traffic is ≈0.7 GB/frame, ≈67% of the A14's bandwidth — §6.5's invariant again). And the 10-minute *foreground* sustain gate currently fails on the A17 Pro: both arms of a controlled soak bank ≈20 s of ring lookahead and then thermal-throttle to ≈0.95× real time around minutes 5–7, screen on, with the UI's Metal load included. The earlier 10-minute zero-underrun runs (headless host, all-ANE placement, pre-fix producer) validated the delivery architecture — SPSC ring, reservoir, backpressure — but not the corrected pipeline's thermal envelope. The named open work is exactly what the findings predict: re-land the temporal stack on the ANE via the stateless host-cache boundary (§6.3), and cut temporal weight bytes (§6.5). We report the gap rather than the trend line.

7 Discussion

7.1 Why Does the Kokoro Pipeline Win?

The Surgical pipeline's advantage over GPU-only frameworks has six sources:

1. **A better scheduling unit.** The original model is not the natural unit of deployment. It contains several computational regimes with different hardware affinities. Once those regimes become separate artifacts, the runtime can schedule them independently instead of forcing the whole graph through one backend.

2. **Ahead-of-time compiled static graphs.** Each Core ML stage is a statically compiled fp16 graph (fixed-shape except the GPU generator's bounded `RangeDim` axis), compiled and fused once at load. PyTorch MPS dispatches Metal kernels op by op from Python. MLX evaluates

lazily and supports graph compilation (`mx.compile`), so the gap to MLX is not simply eager-versus-compiled, and we have not isolated it. Either way, the generator runs on CPU+GPU through Core ML — the *same* silicon — and still beats both Metal baselines.

3. **Elimination of Python.** PyTorch MPS incurs interpreter cost between operations; five `model.forward()` calls through the GIL cost real milliseconds. The Surgical pipeline is Swift end-to-end.

4. **Avoidance of MPS fallback overhead.** PyTorch MPS lacks certain operations (notably `aten::angle`, used in STFT), forcing per-op CPU round-trips. The Surgical pipeline implements hn-nsf natively in Swift and never pays this cost.

5. **Cross-unit concurrency.** Because the stages are separate artifacts with explicit dataflow, the CPU-bound hn-nsf source runs on a background thread *during* the DecoderPre Core ML predict (§4.1). A monolithic graph executes serially on whatever unit the scheduler picked; the decomposition turns one of the largest CPU stages into partly hidden time.

6. **ANE participation where it pays.** Only DecoderPre ships on the ANE (`.cpuAndNeuralEngine`), as a single policy for every machine, for simplicity. Per-machine sweeps show the ANE also wins some Duration and F0Ntrain buckets on an M1, by a few milliseconds each. This is a real but partial contribution.

This is also why the methodology matters for concurrent workloads. A GPU-only Kokoro occupies the GPU for the entire synthesis path. Surgical Inference still uses the GPU for stages that measure best there, but it shortens the total wall time, moves some work to CPU/ANE, exposes idle windows, and avoids routing irregular DSP through the GPU. That is the path to keeping the GPU available for another local workload such as an LLM. We have not yet run a co-scheduling benchmark with TTS and an LLM; the claim here is architectural headroom, not a measured throughput result. Case Study 2 supplies a direct measurement of the underlying property for its earlier all-ANE configuration: the generating process showed zero attributed Metal GPU intervals and zero process GPU power impact while streaming audio (§6.7). The corrected pipeline that ships today runs its temporal stage on the GPU.

We did not run controlled ablations isolating each factor's contribution — and the cleanest such ablation, a monolithic Core ML baseline, is one we could not build for this model (§5.9): conversion fails at the alignment seam, and the strongest partial monolith is numerically invalid. The per-stage compute-unit ablation data (which motivated the staged policy) is the closest available evidence. From it we believe factors 1–5 dominate, but that is a hypothesis, not a measurement. We state the missing factor-isolation as a limitation (§7.5), but the §5.9 control also sharpens the paper's central finding: **the win comes from decomposition and native orchestration, not from any single accelerator — and for this model, decomposition was not merely faster but the precondition for any correct Core ML artifact we could build.**

7.2 The ANE Admission Findings

An earlier draft of this work assumed the Dense-Static stages would run on the ANE under `compute_units=ALL`. The evidence says otherwise, and the correction matters for anyone applying this methodology:

- **The vocoder categorically exceeds ANE tensor limits.** GeneratorFromHar's harmonic input has a last-axis extent of 14,401–144,001 elements across the buckets and its waveform output 72,000–720,000 — far beyond the ≈16,384 per-axis limit documented by the community and

confirmed by our own compiler errors (`Tensor width goes beyond limit supported (16390 > 16384)`).

- **Forcing it fails loudly on every machine; `.all` fails silently.** Loading the generator with `.cpuAndNeuralEngine` fails ANE compilation (`MILCompilerForANE ... ANECCompile() FAILED`) on every Mac tested, and in an April 2026 ablation with the pre-fix generator geometry, the fallback execution ran ≈54× slower than CPU+GPU (≈1517 ms vs ≈28 ms on M2 Ultra, 3 s bucket).[1] Under `.all`, macOS silently reroutes the rejected subgraph — the model "works" and the developer learns nothing. iOS was stricter: on iOS 26.5, `.all` hard-failed at predict time on both test iPhones (on iOS 27.0.1 it runs; §5.6).
- **The shipped policy is therefore explicit and staged**: DecoderPre on `.cpuAndNeuralEngine`; Duration, F0Ntrain, and GeneratorFromHar on `.cpuAndGPU`. The same compute policy runs on Mac and iPhone, so the cross-device numbers are policy-matched (the generator package differs: fixed buckets on the Macs, flexible on the iPhones).
- **Surface-level graph surgery did not change placement.** We tested rewrites suggested by ANE reverse-engineering literature — `nn.Linear` → 1×1 `Conv1d`, native instance-norm lowering, broadcast-AdaIN reformulation, fp16 input dtypes, palettization — against the fused generator. All passed strict numerical validation; none produced a material runtime improvement, and the Linear→Conv1d change was reverted. The MIL compiler appears to make these lowering decisions itself; the binding constraint is tensor geometry, not op selection.

The generalizable lesson: *Dense-Static is the candidate set for the ANE, not a guarantee.* Audio-rate tensors need either re-chunking along the time axis to fit admission limits — future work — or the honest fallback this paper ships: Core ML-compiled CPU+GPU execution, which is already faster than the eager GPU frameworks.

Across the two case studies, three *distinct* binding constraints governed ANE admission, and we found none of them documented as such by the vendor:

1. **Tensor geometry** (Case Study 1): waveform-rate axes exceed per-axis limits; no op rewrite helps because the constraint is shape, not lowering. Case Study 2 routed around this by construction — its codec graph stops at STFT-rate features and the host synthesizes PCM.
2. **In-graph state mutation** (Case Study 2, §6.3): every temporal graph that mutated K/V cache state inside Core ML failed `ANECCompile`; the identical math as a stateless step function compiled clean and beat CPU and GPU. The escape is a boundary redesign, not an op substitution.
3. **FP16 lowering paths coupled to layout** (Case Study 2, §6.4): the same FP16 program produced finite output on the ANE and non-finite output on CPU and GPU after a channels-first rewrite — and was non-finite everywhere before it. Admission here is not binary compile success but numerical survival, per compute unit.

The two case studies also disagree instructively about rewrites: Kokoro's op-level rewrites (Linear → 1×1 conv, norm lowerings) changed nothing, while MRT2's layout rewrite changed everything. Both are consistent with one rule: the MIL compiler already makes op-level lowering decisions, so source cosmetics are no-ops, but *structural* properties — shapes, state boundaries, memory layout — are the developer's lever, and only a per-stage on-device ablation reveals which structural property is binding.

[1] An isolated single-model `predict()` benchmark used for this placement ablation. It is distinct from the ≈2 ms generator figure in §5.7, which is the generator's marginal share of end-to-end wall time, obtained by subtracting the other three models' costs from the observed pipeline total.

7.3 When Surgical Inference Is Worth the Effort

Surgical Inference required substantial engineering: per-bucket Core ML exports with numerical validation, a Swift package with custom DSP, per-stage compute-unit ablations, and a counterbalanced bakeoff harness. This effort is justified when:

- **The workload is compute-bound and parallelizable.** Dense convolutional vocoders, image denoisers, and speech encoders fit. Memory-bandwidth-bound autoregressive transformers do not.
- **The pipeline has distinct stages.** A single undifferentiated block offers fewer decomposition opportunities. Models with encoders, decoders, samplers, DSP, token logic, or post-processors are better candidates.
- **Target hardware spans a range of devices.** The advantage matters most on constrained hardware: PyTorch MPS fails the 15 s and 30 s inputs on every Mac tested, and MLX is jetsam-killed on a 4 GB phone, where the Core ML pipeline completes.
- **Real-time or interactive constraints matter.** For offline batch synthesis on a workstation, PyTorch MPS may be sufficient. For an app that must synthesize speech while rendering UI or running an assistant model, freeing GPU time matters. Case Study 2 is the limiting case: under a hard streaming deadline, per-stage placement and host-owned glue were how we met the deadline on the A17 Pro, not an optional optimization.

The methodology is less useful for:

- LLM inference (memory-bandwidth-bound; the ANE provides no bandwidth advantage — and Case Study 2's bandwidth invariant, §6.5, quantifies exactly why)
- Small one-shot models where Python overhead is already negligible
- Research contexts where PyTorch flexibility outweighs deployment performance

7.4 Applicability to Other Workloads

The empirical claims of this paper now cover two structurally different generative systems: a feed-forward batch TTS pipeline (Kokoro-82M) and a hard-real-time autoregressive streaming music model (MRT2). The second port is evidence that the decomposition procedure (§3.2) is not TTS-specific: the same motif classification, the same per-stage ablation, and the same host-glue discipline produced a working pipeline in a regime with no buckets, no batch boundary, and a stateful transformer on the hot path. Beyond these two:

- **Speech and audio.** Other TTS architectures, neural codecs, source-filter vocoders, speech enhancement, and Whisper-style encoder-decoders all mix dense neural blocks with DSP, masking, or sequence logic. The ANE admission analysis (§7.2) transfers directly: vocoders that emit audio-rate tensors need time-axis chunking or host-side synthesis from feature-rate tensors (the route Case Study 2 took), and streaming encoders with cache state should be exported as stateless step functions (§6.3).
- **Image and video generation.** Encoders, denoisers, VAE decoders, schedulers, safety checkers, and post-processing often have different hardware affinities. Dense fixed-shape blocks are natural Core ML candidates; schedulers and dynamic post-processing belong in native code.

- **Agentic local assistants.** A local LLM may remain on GPU because it is memory-bandwidth-bound, while speech I/O, wake-word models, embedding models, rerankers, image encoders, and audio decoders can be decomposed away from the GPU. The useful question is not "can the LLM run on ANE?" but "which surrounding stages can leave the GPU alone?" Case Study 2's power data (§6.7) shows a nontrivial generative workload streaming with zero process GPU impact, in its earlier all-ANE configuration.

7.5 Limitations

Residency and power instrumentation is asymmetric across case studies. Case Study 1's published runs have no Instruments Core ML timelines or power traces; its staged policy states placement *intent* (the iPhone `.all` rejection on iOS 26.5 indicates the scheduler honors stage policies), but the fraction of DecoderPre actually executing on ANE silicon is unmeasured. Case Study 2 closes this gap for its own pipeline — per-model ANE hardware intervals, app-attributed-GPU-absence checks, and paired Power Profiler captures (§6.7) — and that tooling should be back-ported to the Kokoro benchmarks.

Case Study 2's power comparison predates its correctness fixes. The paired ANE-vs-GPU power captures ran identical graphs in both arms, so the policy-controlled deltas (GPU impact removed, CPU instructions −56%) stand, but the absolute duty cycles were measured on a producer later found semantically incorrect (§6.6) and do not transfer to the corrected pipeline.

Case Study 2's sustained envelope is open. The corrected pipeline is faster than real time on the A17 Pro over bounded runs but fails the 10-minute foreground soak by thermal throttling to ≈0.95× real time (screen on, UI Metal load included), and the A14 is above the frame budget entirely (≈50 ms/frame). The named fixes — re-landing the temporal stack on the ANE via the stateless boundary, and cutting temporal weight bytes — are motivated by the §6.3/§6.5 findings but not yet demonstrated. ANE admission also proved *instance-fragile*: an artifact that compiled to the ANE in a test harness later fell back to CPU inside the shipping app (§6.3), so placement must be re-proven per process, not per artifact.

No concurrent workload benchmark. We have not yet measured synthesis while a local LLM runs on the GPU. The paper's GPU-headroom argument follows from shorter wall time, staged placement, partial CPU/ANE execution, and Case Study 2's zero-GPU-impact traces of its earlier all-ANE configuration, but it needs a direct co-scheduling benchmark before becoming a throughput claim.

MPS fallback overhead in Config D. Config D uses `PYTORCH_ENABLE_MPS_FALLBACK=1`, the realistic developer default, so its timings include CPU fallback costs for unsupported ops. The MLX comparison (§5.3) partially addresses this — MLX is a fallback-free Metal path and the Surgical pipeline still wins by 1.8–2.4× — but a fallback-free MPS variant was not benchmarked.

Mixed OS and a mild-load laptop. Each Mac's results come from one session, but not one OS: the M2 Ultra ran macOS 26.7.1 and the M2 Air and M1 Mini ran macOS 27.0.1. The M2 Air never cleared the quiet-host gate (load 2.2–2.9), so its rows are mild upper bounds (§5.1).

An OS regression shapes the shipped configuration. The library's flexible-length generator, which avoids paying for bucket padding, can abort the process on macOS 27.0.1 inside MPSGraph; the abort is intermittent and cannot be caught. The runtime therefore uses the fixed-bucket generators on OS 27, and the main tables use them on every Mac for comparability.

Remaining frontier on low-end hardware. In June 2026, laishere's chain-only Core ML numbers were faster than our full-pipeline numbers on the M1 Mini's short and medium buckets. That comparison cannot be repeated on macOS 27.0.1, where laishere's converter fails (§5.5). On the M2 Ultra the Surgical pipeline is now 1.9–2.0× faster. On base chips our generator stage

dominates wall time (§5.7), and that is where any remaining gap would be.

Audio quality evaluation is automated, not human, and has not been rerun on the current pipeline. In July 2026, to move beyond informal listening checks, we ran a structured perceptual evaluation of Case Study 1's then-shipped pipeline: the PyTorch eager reference (float32, CPU) versus the Swift + Core ML pipeline under its staged compute policy, judged by a multimodal audio model (Gemini) in blind paired lineups with a known-bad control and repeat-vote agreement, alongside the objective waveform gate. It used four frozen inputs (2.8–27.4 s); the benchmarks in §5 use five, and the 10 s input was never listening-tested. The initial run passed the short and medium buckets cleanly — the judge rated the Core ML output "perceptually indistinguishable" from the reference while unambiguously rejecting a speech-shaped static control that the objective waveform gate cannot catch — and failed both long buckets: 3/3 votes on "unnatural silent gaps" at 15 s, and 2/2 votes on "background static and hiss" at 30 s, backed by signal analysis (phrase pauses measured 40–110 ms long; 1–9 kHz energy at 0.35–0.48× of the gain-matched reference). Stage-boundary tensor bisection traced both to implementation defects invisible to per-stage matched-input correlation: a native post-process that silenced punctuation-adjacent whitespace spans carrying real speech (−11 to −22 dBFS word onsets and phrase-final decays), and an export-time inverse-STFT substitution that scaled every interior frequency bin of the one-sided spectrum at half amplitude — a defect present in *every* bucket and every compute policy, which the judge only flagged where exposure was longest. A third, compounding defect surfaced during root-causing rather than from the perceptual gate directly: the Swift harmonic-source noise generator's xorshift64 RNG had an absorbing zero state, so the fixed seed used for every evaluated clip produced a degenerate DC-and-Nyquist-only "noise" signal instead of broadband Gaussian noise. After fixing all three (suppression narrowed to punctuation tokens; corrected one-sided iSTFT weights re-exported, matching the reference generator to 5×10^{-7} max abs; the RNG reseeded via SplitMix64 scrambling so no seed reaches the absorbing state), all four inputs passed the blind evaluation (15 s 2/2, 30 s 2/2) and the control was still rejected.

Three later findings changed the pipeline that evaluation certified. The punctuation-narrowing "fix" was itself found to cut speech — PyTorch renders −8 to −25 dBFS onsets and decays inside punctuation-owned spans — and the fade was removed. Bucket padding had contaminated normalization statistics: with one 2.8 s utterance forced through every bucket, F0 sat at 15 dB SNR against a native-length run even at 93% fill, and the decoder output at −4.7 dB at 9% fill; masking fixed it (§4.2). And the Swift STFT reported $-\pi$ where PyTorch reports $+\pi$ at the Nyquist bin, in 10,658 frames of the frozen 15 s input, which put +1 to +3 dB of excess energy above 6 kHz into every earlier render. All three are fixed, but the blind evaluation has not been rerun on the current pipeline. It is also a model-judged evaluation, not a human MOS study: it demonstrated the discriminative power to catch shipped defects, but it does not calibrate absolute quality scores, and its pass verdicts apply to the July pipeline. Commands, verdicts, signal analyses, and the root-cause chain are recorded in the repository (Appendix A). Case Study 2 reports token-level parity, embedding-space adherence and stereo-correlation metrics against the MLX reference, and blind automated listening gates with known-bad controls — also short of a formal MOS study, and its adherence scores sit below the upstream runtime-CFG reference because the on-device pipeline bakes guidance at export time.

Single-vendor evaluation. All measurements are on Apple Silicon. The motif taxonomy and the decomposition procedure do not assume Core ML, but the constraint catalog (§7.2) is Apple-specific; other NPU toolchains (Qualcomm QNN, LiteRT delegates) impose different admission rules and would need their own per-stage ablations before the quantitative claims transfer.

Hardware coverage. Macs: M1, M2, M2 Ultra; iPhones: A14, A17 Pro (both case studies use the same two phones). M3/M4 Macs and A18/A19-class phones were not tested.

8 Related Work

Core ML deployment of large models. Apple has published a Core ML conversion of Stable Diffusion [1], which already ships the pipeline as separate Core ML packages per model (text encoder, UNet, VAE decoder), and vendor guidance for ANE-friendly transformer deployment [6]; the latter documents channels-first layout conventions as a throughput optimization, which our §6.4 finding extends to a numerical-survival condition. In the speech domain, whisper.cpp [12] added a Core ML encoder path alongside its Metal decoder, and WhisperKit [28] decomposes Whisper into per-stage Core ML models with per-stage compute units and ships them in production on Apple devices. These are close precedents, and Surgical Inference systematizes their practice rather than inventing it. What this paper adds is an explicit cross-model procedure (motif classification, per-stage placement ablation, host-glue validation rules), the measured admission findings that make the ablation necessary, the negative results, and two end-to-end pipelines on structurally different generative workloads.

Live music generation on device. Magenta RealTime and its successor MRT2 [23, 25] are, to our knowledge, the first open-weights live-music models with a real-time streaming contract; the upstream deployment path is MLX on M-series Macs, with desktop standalone and plugin surfaces. We are not aware of prior work running this model class in real time on a phone, nor of prior documentation of the state-mutation ANE admission cliff (§6.3) or the layout-dependent FP16 survival result (§6.4) that doing so surfaced; Apple's stateful-model documentation [2] describes the `MLState` API but does not specify ANE admission behavior for graphs that mutate state in-graph. The SpectroStream codec follows the residual-vector-quantization lineage of SoundStream [36].

Core ML Kokoro ports. laishere/kokoro-coreml [21] runs a seven-package ANE-resident Kokoro vocoder chain and is the strongest prior art for this specific model; we benchmark against it directly (§5.5) and adopt its chain-only boundary caveat when comparing. soniqo/speech-swift [33] ships a Core ML Kokoro for iOS with a fixed-length output artifact. mlalma/kokoro-ios [26] is the MLX Swift port we compare against on-device (§5.6).

On-device TTS. Piper [29] targets edge deployment with CPU-only inference. MeloTTS [27] provides real-time multilingual synthesis on CPU. These systems prioritize portability across CPUs; we did not benchmark them.

MLX for Apple Silicon. MLX [13] provides a NumPy-like GPU interface for Apple Silicon. MLX is complementary to our work: Sequential-Dynamic stages of a larger pipeline could use MLX while Dense-Static stages use Core ML. Our measurements (§5.3, §5.6) characterize where the Core ML path wins for convolutional TTS today, including the memory-bounded regime where MLX fails outright.

Reverse engineering the ANE. The Orion project [20] and community field guides [32] document reverse-engineered ANE constraints (per-dispatch overheads, tensor geometry limits, preferred lowerings). Our experiments corroborate the geometry limits as binding for audio-rate tensors, and find that source-level op rewrites suggested by this literature (e.g. Linear $\rightarrow$ 1×1 conv) do not change Core ML runtime performance — the MIL compiler already makes those decisions (§7.2). Case Study 2 adds two constraints we did not find in this literature: in-graph state mutation as a compile-time admission cliff for streaming transformers, and FP16 numerical survival that differs across compute units as a function of internal layout (§7.2).

Heterogeneous accelerator scheduling. The problem of mapping computation to heterogeneous accelerators has been studied extensively in CPU/GPU systems (StarPU [7], Legion [8]) and data-center contexts (TPU/GPU/CPU). In mobile deployment, delegate-based runtimes (ExecuTorch [30], LiteRT/NNAPI) partition a single graph across NPU backends at runtime. A line of mobile-systems work schedules inference across a phone's processors directly: μLayer [19]

splits single layers across CPU and GPU, CoDL [17] co-executes DNN inference on CPU and GPU, and Band [16] coordinates multiple DNNs across heterogeneous mobile processors; for LLMs, llm.npu [34] offloads prefill to the NPU, and PowerInfer-2 [35] and HeteroLLM [10] split inference across a smartphone's heterogeneous processors. These systems schedule DNN or LLM operators on Android phones, mostly Qualcomm SoCs, with programmable access to each processor. This paper studies a different constraint surface: Apple's Core ML, whose ANE admission is opaque and must be probed per stage, and the decomposition of generative audio pipelines into separately compiled stages whose partition is an explicit, measured design artifact rather than a runtime decision.

9 Conclusion

We presented Surgical Inference, a methodology for decomposing generative models into smaller neural submodels and native stages that can be scheduled across a heterogeneous SoC, and evaluated it on two structurally opposite workloads. By classifying stages into three computational motifs — Sequential-Dynamic, Data-Dependent Logic, and Dense-Static — matching each to candidate hardware, and resolving final placement with per-stage measurement, we produced a Kokoro-82M pipeline that is 1.8–2.4× faster than MLX, 2.7–6.5× faster than PyTorch MPS where MPS completes at all, and 3.2–6.8× faster than PyTorch CPU, across three Macs spanning the consumer-to-workstation range. The same model families deploy to iPhone, where the pipeline beats the MLX Swift port by 2.0–3.2× and completes the 30 s input that the MLX port cannot load on a 4 GB phone.

The same procedure carried Magenta RealTime 2 — a 230M-parameter autoregressive streaming music model whose upstream real-time path requires an M-series Mac GPU — onto iPhone hardware, and sharpened the methodology along the way: the Sequential-Dynamic motif is divisible into Dense-Static math and host-owned cache bookkeeping, layout can decide FP16 survival per compute unit, and weight bandwidth sets the cost of each call on bandwidth-bound stages (§7.2, §6.5). Residency traces and paired power captures of the earlier all-ANE configuration showed zero app-attributed GPU time. The corrected pipeline, which runs the temporal stage on the GPU and the depth stage on the CPU, is faster than real time on an iPhone 15 Pro Max; the 10-minute thermal envelope and the A14 budget remain open, and we report them as open.

The "use the GPU" convention in local inference is suboptimal for compute-bound generative pipelines with distinct stages, and the inverse oversimplification fails too: the Neural Engine is not a substrate that dense graphs simply run on. In both case studies the win comes from decomposition — static per-stage graphs, native orchestration, deliberate placement, and validated host glue — with each accelerator contributing where its admission constraints allow. The right question for on-device AI is not "which single accelerator should run this model?" but "what is the smallest set of static neural packages and native kernels that lets the whole chip execute the workload?" — for batch synthesis on hardware people already own, and for generative audio that streams in real time from the phone in their pocket.

Acknowledgments

Several improvements to the Kokoro pipeline reported here came from outside contributors to the open-source repository. Willem Kokke contributed mask-aware bucketing, which replaced the duration model's unrolled BiLSTM with stock LSTM operations and stopped bucket padding from biasing normalization statistics. He also removed the punctuation-span fade, corrected the generator's harmonic-input geometry and the STFT phase sign at the DC and Nyquist bins, ran the

harmonic source concurrently, and contributed the flexible-length generator. Ahmed El Hadidi fixed a Swift error-mapping build failure, and David Bitton fixed the Intel (x86_64) SDK build. We thank them for their work and their careful measurements.

References

[1] Apple. Stable diffusion with core ml on apple silicon. https://github.com/apple/ml-stable-diffusion, 2022.

[2] Apple. MLState — core ml stateful models documentation. https://developer.apple.com/documentation/coreml/mlstate, 2024.

[3] Apple. Core ml tools. https://github.com/apple/coremltools, 2025. Software package for converting trained models to Core ML; version 8.3.0 (April 2025) used for every export.

[4] Apple. Pytorch conversion workflow. https://apple.github.io/coremltools/docs-guides/source/convert-pytorch-workflow.html, 2026. Core ML Tools guide. Accessed October 2026.

[5] Apple Developer. Accelerated pytorch training on mac. https://developer.apple.com/metal/pytorch/, 2026. PyTorch MPS backend documentation. Accessed October 2026.

[6] Apple Machine Learning Research. Deploying transformers on the apple neural engine. https://machinelearning.apple.com/research/neural-engine-transformers, June 2022. Reference implementation: https://github.com/apple/ml-ane-transformers.

[7] Cédric Augonnet, Samuel Thibault, Raymond Namyst, and Pierre-André Wacrenier. StarPU: A unified platform for task scheduling on heterogeneous multicore architectures. *Concurrency and Computation: Practice and Experience*, 23(2):187–198, 2011.

[8] Michael Bauer, Sean Treichler, Elliott Slaughter, and Alex Aiken. Legion: Expressing locality and independence with logical regions. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC)*, 2012.

[9] Blaizzy. mlx-audio. https://github.com/Blaizzy/mlx-audio, 2026. Version 0.4.3, commit 862dfbe (June 2026); MLX audio generation toolkit used for the MLX Kokoro baseline.

[10] Le Chen, Dahu Feng, Erhu Feng, Yingrui Wang, Rong Zhao, Yubin Xia, Pinjie Xu, and Haibo Chen. Characterizing mobile SoC for accelerating heterogeneous LLM inference. https://arxiv.org/abs/2501.14794, 2025. Version 1 titled "HeteroLLM: Accelerating Large Language Model Inference on Mobile SoCs platform with Heterogeneous AI Accelerators"; the engine is called HeteroInfer in later versions.

[11] Georgi Gerganov and contributors. llama.cpp. https://github.com/ggml-org/llama.cpp, 2026. Software. Accessed October 2026.

[12] Georgi Gerganov and contributors. whisper.cpp. https://github.com/ggml-org/whisper.cpp, 2026. Software; includes a Core ML encoder path. Accessed October 2026.

[13] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on apple silicon. https://github.com/ml-explore/mlx, 2023. Software.

[14] hexgrad. Kokoro-82m. https://huggingface.co/hexgrad/Kokoro-82M, 2024. Open-weight 82M-parameter text-to-speech model; model repository created December 2024.

[15] hexgrad. kokoro. https://github.com/hexgrad/kokoro, 2025. Python inference library for Kokoro-82M; repository created January 2025.

[16] Joo Seong Jeong, Jingyu Lee, Donghyun Kim, Changmin Jeon, Changjin Jeong, Youngki Lee, and Byung-Gon Chun. Band: Coordinated multi-DNN inference on heterogeneous mobile processors. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services (MobiSys)*, pages 235–247, 2022. doi: 10.1145/3498361.3538948.

[17] Fucheng Jia, Deyu Zhang, Ting Cao, Shiqi Jiang, Yunxin Liu, Ju Ren, and Yaoxue Zhang. CoDL: Efficient CPU-GPU co-execution for deep learning inference on mobile devices. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services (MobiSys)*, pages 209–221, 2022. doi: 10.1145/3498361.3538932.

[18] Takuhiro Kaneko, Kou Tanaka, Hirokazu Kameoka, and Shogo Seki. iSTFTNet: Fast and lightweight mel-spectrogram vocoder incorporating inverse short-time fourier transform. In *ICASSP*, 2022. https://arxiv.org/abs/2203.02395.

[19] Youngsok Kim, Joonsung Kim, Dongju Chae, Daehyun Kim, and Jangwoo Kim. μLayer: Low latency on-device inference using cooperative single-layer acceleration and processor-friendly quantization. In *Proceedings of the Fourteenth EuroSys Conference (EuroSys)*, pages 1–15, 2019. doi: 10.1145/3302424.3303950.

[20] Ramchand Kumaresan. Orion: Characterizing and programming apple's neural engine for LLM training and inference. https://arxiv.org/abs/2603.06728, 2026.

[21] laishere. kokoro-coreml. https://github.com/laishere/kokoro-coreml, 2026. Commit 484907d; independent Core ML Kokoro implementation used as a comparison baseline. Accessed October 2026.

[22] Yinghao Aaron Li, Cong Han, Vinay S. Raghavan, Gavin Mischler, and Nima Mesgarani. StyleTTS 2: Towards human-level text-to-speech through style diffusion and adversarial training with large speech language models. In *NeurIPS*, 2023. https://arxiv.org/abs/2306.07691.

[23] Lyria Team, Antoine Caillon, Brian McWilliams, Cassie Tarakajian, Ian Simon, Ilaria Manco, Jesse Engel, Noah Constant, et al. Live music models. https://arxiv.org/abs/2508.04651, 2025. Introduces Magenta RealTime.

[24] Chris J. Maddison, Daniel Tarlow, and Tom Minka. A* sampling. In *NeurIPS*, 2014. https://arxiv.org/abs/1411.0030.

[25] Magenta team, Google. magenta-realtime. https://github.com/magenta/magenta-realtime, 2026. Open-weights live music model repository; upstream source for the second case study. Accessed October 2026.

[26] mlalma. kokoro-ios. https://github.com/mlalma/kokoro-ios, 2025. Version 1.0.8 (October 2025); MLX Swift Kokoro implementation used for iOS comparison.

[27] MyShell.ai. MeloTTS. https://github.com/myshell-ai/MeloTTS, 2024. Open-source multilingual text-to-speech system; repository created February 2024.

[28] Atila Orhon et al. WhisperKit: On-device real-time ASR with billion-scale transformers. https://arxiv.org/abs/2507.10860, 2025. Software: https://github.com/argmaxinc/WhisperKit.

[29] Piper contributors. Piper. https://github.com/rhasspy/piper, 2023. Fast local neural text-to-speech system; repository created January 2023, since archived.

[30] PyTorch. ExecuTorch. https://github.com/pytorch/executorch, 2026. On-device inference runtime with delegate backends. Accessed October 2026.

[31] PyTorch. MPS backend notes. https://pytorch.org/docs/stable/notes/mps.html, 2026. Apple Silicon GPU backend documentation. Accessed October 2026.

[32] skyfallsin. Apple neural engine reverse-engineering field guide. https://github.com/skyfallsin/apple-neural-engine-field-guide, 2026. Experiment-driven ANE notes and constraints. Accessed October 2026.

[33] soniqo. speech-swift. https://github.com/soniqo/speech-swift, 2026. Commit 0d09a2e; independent Core ML Kokoro implementation used as a comparison baseline. Accessed October 2026.

[34] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. Fast on-device LLM inference with NPUs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 445–462, 2025. doi: 10.1145/3669940.3707239.

[35] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. PowerInfer-2: Fast large language model inference on a smartphone. https://arxiv.org/abs/2406.06282, 2024.

[36] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. SoundStream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30:495–507, 2022. doi: 10.1109/TASLP.2021.3129994. https://arxiv.org/abs/2107.03312.

A Artifact and Reproduction Map

A.1 Case Study 1 (Kokoro-82M)

The implementation artifacts for the first case study are in the open-source `kokoro-coreml` repository:

- Swift runtime pipeline: `swift/Sources/KokoroPipeline/`
- Main pipeline loader and staged compute-unit policy: `swift/Sources/KokoroPipeline/KokoroPipeline.swift`
- End-to-end synthesis executor and DecoderPre/hn-nsf overlap: `swift/Sources/KokoroPipeline/KokoroSynthesisExecutor.swift`
- Direct token-to-frame expansion: `swift/Sources/KokoroPipeline/MLMultiArrayHelpers.swift`

- Harmonic source DSP implementation: `swift/Sources/KokoroPipeline/HarmonicSource.swift`
- Waveform trim: `swift/Sources/KokoroPipeline/KokoroSynthesisExecutor.swift`
- Mac bakeoff harness: `scripts/bakeoff_harness.py`
- External bakeoff adapters: `scripts/external_bakeoff/`
- iPhone benchmark app: `ios-bench/`
- Monolithic-export control experiment (§5.9), protocol and verbatim logs: `README/Notes/monolithic-coreml-control-experiment.md`
- Structured perceptual evaluation (§7.5), protocol, verdicts, and signal analyses: `README/Notes/cs1-audio-quality-evaluation-2026-07-14.md`
- October 2026 iPhone data (§5.6): `README/Notes/paper-cs1-frozen-rerun-2026-10-20.md`; thermal protocol and Debug-vs-Release methodology from the July 2026 runs: `README/Notes/iphone-release-build-mlx-comparison.md`
- October 2026 Mac benchmark run (§5.1–§5.8), protocol, quiet-host log, and result summary: `README/Notes/paper-cs1-frozen-rerun-2026-10-20.md` (plan: `README/Plans/013-paper-cs1-frozen-rerun-plan.md`)

The pre-converted Core ML packages are published with the repository release artifacts and on the project Hugging Face model page.

A.2 Case Study 2 (Magenta RealTime 2 / Crossfade)

The conversion and validation layer is public in the `magenta-realtime-2-iphone` repository and covers both artifact generations:

- Corrected exporters reproducing the §6.3–§6.5 findings: `exporters/convert_temporal_body_carry.py` (stateless host-owned-cache temporal boundary, §6.3), `exporters/convert_depth_body_rollout.py` (in-graph FP16 depth rollout, §6.5), and `exporters/convert_spectrostream_decoder.py--fp16-rescale` (the finite-FP16 channels-first decoder path, §6.4)
- Superseded exporters retained and labeled as negative-result artifacts: running the stateful temporal exporter reproduces the `ANECCompile` −14 rejection, and the decoder without the FP16-safe path reproduces the non-finite output
- Numerical-parity validators and validation receipts: `validation/` (including the depth rollout's 0/900 token-exactness receipt)
- Reference fixtures, graph-teardown and KV-cache documentation, and the artifact inventory with tensor contracts: `fixtures/`, `docs/`, `MODELS.md`

Pre-built `.mlpackage` binaries on the Hugging Face mirror are the superseded generation at the time of writing; the corrected artifacts are reproducible from the exporters above, and `MODELS.md` states per artifact what is uploaded versus reproducible-from-exporter. The streaming runtime and instrumentation live in the `crossfade` repository, a fork of `magenta/magenta-realtime` that is private at the time of writing (artifacts available from the author). Within it:

- Conversion wrappers and exporters: `magenta_rt/coreml/`, `scripts/convert_mrt2_*.py`, `scripts/convert_spectrostream_decoder_*.py`
- Numerical-parity validators (vs the MLX reference): `scripts/validate_mrt2_*.py`, `scripts/verify_spectrostream_streaming_decode.py`
- On-device probe used for the falsification ladder and placement matrices: `examples/ios/CoreMLMRT2Probe/`
- Production runtime (stateless temporal boundary, depth rollout, SPSC ring, reservoir/backpressure): `Sources/CrossfadeRuntime/`, `Sources/CrossfadeRuntimeCore/`
- Residency-proof runner emitting the machine-checkable `placement-evidence.json`: `scripts/run_coreml_trace_device.py` with `scripts/summarize_coreml_xctrace_exports.py`
- Paired ANE-vs-GPU power capture with the comparability gate: `scripts/run_power_profiler_pair_device.py` with `scripts/summarize_power_xctrace_exports.py`
- Sustained-run log analyzer (duty cycle, backpressure share, underruns, thermal): `scripts/analyze_crossfade_runtime_host_log.py`
- Investigation ledgers with per-run device log paths: `README/Notes/mrt2-coreml-proof-v1.md`, `README/Notes/mrt2-small-graph-teardown-v1.md`

B Benchmark Boundaries

The benchmark boundary used throughout Case Study 1 is:

```
phoneme token IDs + voice embedding
    -> full synthesis pipeline
    -> 24 kHz PCM waveform resident in memory
```

Model download, process startup, first-use Core ML compilation/cache effects, playback, and text-to-phoneme conversion are excluded unless a comparison implementation's public API forces a wider boundary. Such boundary differences are disclosed in §5.5 and §5.6.

Case Study 2 uses two boundaries appropriate to streaming. Stage and per-frame timings are measured at the Core ML prediction call (warmed, p50/p99 over sustained runs, with `MLComputePlan` or Instruments placement evidence attached to the same run). End-to-end claims are measured at the audio contract: frames pushed into and pulled from the render ring by the live `AVAudioSourceNode`, with underrun events, dropped frames, ring fill watermarks, and `ProcessInfo.thermalState` logged per generation iteration. Sustained results report the full counter set rather than a single latency figure, because a streaming pipeline can have excellent median latency and still fail its product contract at minute seven.